# pytest Documentation

*Release 9.1*

**holger krekel, trainer and consultant, https://merlinux.eu/**

**Jun 19, 2026**

# CONTENTS

Download latest version as PDF

CHAPTER

ONE

# START HERE

## 1.1 Get Started

### 1.1.1 Install `pytest`

1. Run the following command in your command line:

```
pip install -U pytest
```

2. Check that you installed the correct version:

```
$ pytest --version
pytest 9.1.1
```

### 1.1.2 Create your first test

Create a new file called `test_sample.py`, containing a function, and a test:

```
# content of test_sample.py
def func(x):
    return x + 1


def test_answer():
    assert func(3) == 5
```

The test

```
$ pytest
========================== test session starts ==========================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_sample.py F                                                  [100%]

================================ FAILURES ================================
_____________________________ test_answer _____________________________

    def test_answer():
>       assert func(3) == 5
```

(continues on next page)

(continued from previous page)

```
E       assert 4 == 5
E        +  where 4 = func(3)

test_sample.py:6: AssertionError
========================= short test summary info ==========================
FAILED test_sample.py::test_answer - assert 4 == 5
============================ 1 failed in 0.12s =============================
```

The `[100%]` refers to the overall progress of running all test cases. After it finishes, pytest then shows a failure report because `func(3)` does not return `5`.

**Note**

You can use the `assert` statement to verify test expectations. pytest's Advanced assertion introspection will intelligently report intermediate values of the assert expression so you can avoid the many names of JUnit legacy methods.

### 1.1.3 Run multiple tests

`pytest` will run all files of the form `test_*.py` or `*_test.py` in the current directory and its subdirectories. More generally, it follows *standard test discovery rules*.

### 1.1.4 Assert that a certain exception is raised

Use the *raises* helper to assert that some code raises an exception:

```
# content of test_sysexit.py
import pytest


def f():
    raise SystemExit(1)


def test_mytest():
    with pytest.raises(SystemExit):
        f()
```

Execute the test function with "quiet" reporting mode:

```
$ pytest -q test_sysexit.py
.                                                                    [100%]
1 passed in 0.12s
```

**Note**

The `-q/--quiet` flag keeps the output brief in this and following examples.

See *Assertions about expected exceptions* for specifying more details about the expected exception.

### 1.1.5 Group multiple tests in a class

Once you develop multiple tests, you may want to group them into a class. pytest makes it easy to create a class containing more than one test:

```
# content of test_class.py
class TestClass:
    def test_one(self):
        x = "this"
        assert "h" in x

    def test_two(self):
        x = "hello"
        assert hasattr(x, "check")
```

`pytest` discovers all tests following its *Conventions for Python test discovery*, so it finds both `test_` prefixed functions. There is no need to subclass anything, but make sure to prefix your class with `Test` otherwise the class will be skipped. We can simply run the module by passing its filename:

```
$ pytest -q test_class.py
.F                                                                   [100%]
================================= FAILURES =================================
_____________________________ TestClass.test_two ______________________________

self = <test_class.TestClass object at 0xdeadbeef0001>

    def test_two(self):
        x = "hello"
>       assert hasattr(x, "check")
E       AssertionError: assert False
E        +  where False = hasattr('hello', 'check')

test_class.py:8: AssertionError
========================= short test summary info ==========================
FAILED test_class.py::TestClass::test_two - AssertionError: assert False
1 failed, 1 passed in 0.12s
```

The first test passed and the second failed. You can easily see the intermediate values in the assertion to help you understand the reason for the failure.

Grouping tests in classes can be beneficial for the following reasons:

- Test organization
- Sharing fixtures for tests only in that particular class
- Applying marks at the class level and having them implicitly apply to all tests

Something to be aware of when grouping tests inside classes is that each test has a unique instance of the class. Having each test share the same class instance would be very detrimental to test isolation and would promote poor test practices. This is outlined below:

```
# content of test_class_demo.py
class TestClassDemoInstance:
    value = 0

    def test_one(self):
```

(continues on next page)

(continued from previous page)

```
        self.value = 1
        assert self.value == 1

    def test_two(self):
        assert self.value == 1
```

```
$ pytest -k TestClassDemoInstance -q
.F                                                                   [100%]
================================= FAILURES =================================
_________________________ TestClassDemoInstance.test_two _________________________

self = <test_class_demo.TestClassDemoInstance object at 0xdeadbeef0002>

    def test_two(self):
>       assert self.value == 1
E       assert 0 == 1
E        +  where 0 = <test_class_demo.TestClassDemoInstance object at 0xdeadbeef0002>
↪.value

test_class_demo.py:9: AssertionError
========================= short test summary info ==========================
FAILED test_class_demo.py::TestClassDemoInstance::test_two - assert 0 == 1
1 failed, 1 passed in 0.12s
```

Note that attributes added at class level are *class attributes*, so they will be shared between tests.

### 1.1.6 Compare floating-point values with pytest.approx

`pytest` also provides a number of utilities to make writing tests easier. For example, you can use *`pytest.approx()`* to compare floating-point values that may have small rounding errors:

```
# content of test_approx.py
import pytest


def test_sum():
    assert (0.1 + 0.2) == pytest.approx(0.3)
```

This avoids the need for manual tolerance checks or using `math.isclose` and works with scalars, lists, and NumPy arrays.

### 1.1.7 Request a unique temporary directory for functional tests

`pytest` provides Builtin fixtures/function arguments to request arbitrary resources, like a unique temporary directory:

```
# content of test_tmp_path.py
def test_needsfiles(tmp_path):
    print(tmp_path)
    assert 0
```

List the name `tmp_path` in the test function signature and `pytest` will lookup and call a fixture factory to create the resource before performing the test function call. Before the test runs, `pytest` creates a unique-per-test-invocation temporary directory:

```
$ pytest -q test_tmp_path.py
F                                                                    [100%]
================================= FAILURES =================================
_________________________________ test_needsfiles _________________________________

tmp_path = PosixPath('PYTEST_TMPDIR/test_needsfiles0')

    def test_needsfiles(tmp_path):
        print(tmp_path)
>       assert 0
E       assert 0

test_tmp_path.py:3: AssertionError
--------------------------- Captured stdout call ---------------------------
PYTEST_TMPDIR/test_needsfiles0
========================= short test summary info ==========================
FAILED test_tmp_path.py::test_needsfiles - assert 0
1 failed in 0.12s
```

More info on temporary directory handling is available at *Temporary directories and files*.

Find out what kind of builtin *pytest fixtures* exist with the command:

```
pytest --fixtures   # shows builtin and custom fixtures
```

Note that this command omits fixtures with leading _ unless the `-v` option is added.

### 1.1.8 Continue reading

Check out additional pytest resources to help you customize tests for your unique workflow:

- "*How to invoke pytest*" for command line invocation examples
- "*How to use pytest with an existing test suite*" for working with preexisting tests
- "*How to mark test functions with attributes*" for information on the `pytest.mark` mechanism
- "*Fixtures reference*" for providing a functional baseline to your tests
- "*Writing plugins*" for managing and writing plugins
- "*Good Integration Practices*" for virtualenv and test layouts

CHAPTER

TWO

# HOW-TO GUIDES

## 2.1 How to invoke pytest

**See also**

*Complete pytest command-line flags reference*

In general, pytest is invoked with the command `pytest` (see below for *other ways to invoke pytest*). This will execute all tests in all files whose names follow the form `test_*.py` or `*_test.py` in the current directory and its subdirectories. More generally, pytest follows *standard test discovery rules*.

### 2.1.1 Specifying which tests to run

Pytest supports several ways to run and select tests from the command-line or from a file (see below for *reading arguments from file*).

**Run tests in a module**

```
pytest test_mod.py
```

**Run tests in a directory**

```
pytest testing/
```

**Run tests by keyword expressions**

```
pytest -k 'MyClass and not method'
```

This will run tests which contain names that match the given *string expression* (case-insensitive), which can include Python operators that use filenames, class names and function names as variables. The example above will run `TestMyClass.test_something` but not `TestMyClass.test_method_simple`. Use `""` instead of `''` in expression when running this on Windows

**Run tests by collection arguments**

Pass the module filename relative to the working directory, followed by specifiers like the class name and function name separated by `::` characters, and parameters from parameterization enclosed in `[]`.

To run a specific test within a module:

```
pytest tests/test_mod.py::test_func
```

To run all tests in a class:

```
pytest tests/test_mod.py::TestClass
```

Specifying a specific test method:

```
pytest tests/test_mod.py::TestClass::test_method
```

Specifying a specific parametrization of a test:

```
pytest tests/test_mod.py::test_func[x1,y2]
```

**Run tests by marker expressions**

To run all tests which are decorated with the `@pytest.mark.slow` decorator:

```
pytest -m slow
```

To run all tests which are decorated with the annotated `@pytest.mark.slow(phase=1)` decorator, with the `phase` keyword argument set to `1`:

```
pytest -m "slow(phase=1)"
```

For more information see *marks*.

**Run tests from packages**

```
pytest --pyargs pkg.testing
```

This will import `pkg.testing` and use its filesystem location to find and run tests from.

**Read arguments from file**

Added in version 8.2.

All of the above can be read from a file using the `@` prefix:

```
pytest @tests_to_run.txt
```

where `tests_to_run.txt` contains an entry per line, e.g.:

```
tests/test_file.py
tests/test_mod.py::test_func[x1,y2]
tests/test_mod.py::TestClass
-m slow
```

This file can also be generated using `pytest --collect-only -q` and modified as needed.

### 2.1.2 Getting help on version, option names, environment variables

```
pytest --version   # shows where pytest was imported from
pytest --fixtures  # show available builtin function arguments
pytest -h | --help # show help on command line and config file options
```

### 2.1.3 Profiling test execution duration

Changed in version 6.0.

To get a list of the slowest 10 test durations over 1.0s long:

```
pytest --durations=10 --durations-min=1.0
```

By default, pytest will not show test durations that are too small (<0.005s) unless `-vv` is passed on the command-line.

### 2.1.4 Managing loading of plugins

#### Early loading plugins

You can early-load plugins (internal and external) explicitly in the command-line with the *-p* option:

```
pytest -p mypluginmodule
```

The option receives a `name` parameter, which can be:

- A full module dotted name, for example `myproject.plugins`. This dotted name must be importable.
- The entry-point name of a plugin. This is the name passed to `importlib` when the plugin is registered. For example to early-load the pytest-cov plugin you can use:

  ```
  pytest -p pytest_cov
  ```

#### Disabling plugins

To disable loading specific plugins at invocation time, use the *-p* option together with the prefix `no:`.

Example: to disable loading the plugin `doctest`, which is responsible for executing doctest tests from text files, invoke pytest like this:

```
pytest -p no:doctest
```

### 2.1.5 Other ways of calling pytest

#### Calling pytest through `python -m pytest`

You can invoke testing through the Python interpreter from the command line:

```
python -m pytest [...]
```

This is almost equivalent to invoking the command line script `pytest [...]` directly, except that calling via `python` will also add the current directory to `sys.path`.

#### Calling pytest from Python code

You can invoke `pytest` from Python code directly:

```
retcode = pytest.main()
```

this acts as if you would call "pytest" from the command line. It will not raise `SystemExit` but return the exit code instead. If you don't pass it any arguments, `main` reads the arguments from the command line arguments of the process (`sys.argv`), which may be undesirable. You can pass in options and arguments explicitly:

```
retcode = pytest.main(["-x", "mytestdir"])
```

You can specify additional plugins to `pytest.main`:

```
# content of myinvoke.py
import sys

import pytest


class MyPlugin:
    def pytest_sessionfinish(self):
        print("*** test run reporting finishing")


if __name__ == "__main__":
    sys.exit(pytest.main(["-qq"], plugins=[MyPlugin()]))
```

Running it will show that `MyPlugin` was added and its hook was invoked:

```
$ python myinvoke.py
*** test run reporting finishing
```

**Note**

Calling `pytest.main()` will result in importing your tests and any modules that they import. Due to the caching mechanism of python's import system, making subsequent calls to `pytest.main()` from the same process will not reflect changes to those files between the calls. For this reason, making multiple calls to `pytest.main()` from the same process (in order to re-run tests, for example) is not recommended.

## 2.2 How to write and report assertions in tests

### 2.2.1 Asserting with the `assert` statement

`pytest` allows you to use the standard Python `assert` for verifying expectations and values in Python tests. For example, you can write the following:

```
# content of test_assert1.py
def f():
    return 3


def test_function():
    assert f() == 4
```

to assert that your function returns a certain value. If this assertion fails you will see the return value of the function call:

```
$ pytest test_assert1.py
========================== test session starts ==========================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
```

(continues on next page)

(continued from previous page)

```
collected 1 item

test_assert1.py F                                                    [100%]

================================= FAILURES =================================
_____________________________ test_function ______________________________

    def test_function():
>       assert f() == 4
E       assert 3 == 4
E        +  where 3 = f()

test_assert1.py:6: AssertionError
========================== short test summary info ==========================
FAILED test_assert1.py::test_function - assert 3 == 4
============================ 1 failed in 0.12s =============================
```

`pytest` has support for showing the values of the most common subexpressions including calls, attributes, comparisons, and binary and unary operators. (See *Demo of Python failure reports with pytest*). This allows you to use the idiomatic python constructs without boilerplate code while not losing introspection information.

If a message is specified with the assertion like this:

```
assert a % 2 == 0, "value was odd, should be even"
```

it is printed alongside the assertion introspection in the traceback.

See *Assertion introspection details* for more information on assertion introspection.

### 2.2.2 Assertions about approximate equality

When comparing floating point values (or arrays of floats), small rounding errors are common. Instead of using `assert abs(a - b) < tol` or `numpy.isclose`, you can use *`pytest.approx()`*:

```
import pytest
import numpy as np


def test_floats():
    assert (0.1 + 0.2) == pytest.approx(0.3)


def test_arrays():
    a = np.array([1.0, 2.0, 3.0])
    b = np.array([0.9999, 2.0001, 3.0])
    assert a == pytest.approx(b)
```

`pytest.approx` works with scalars, lists, dictionaries, and NumPy arrays. It also supports comparisons involving NaNs.

See *`pytest.approx()`* for details.

### 2.2.3 Assertions about expected exceptions

In order to write assertions about raised exceptions, you can use *pytest.raises()* as a context manager like this:

```
import pytest


def test_zero_division():
    with pytest.raises(ZeroDivisionError):
        1 / 0
```

and if you need to have access to the actual exception info you may use:

```
def test_recursion_depth():
    with pytest.raises(RuntimeError) as excinfo:

        def f():
            f()

        f()
    assert "maximum recursion" in str(excinfo.value)
```

`excinfo` is an *ExceptionInfo* instance, which is a wrapper around the actual exception raised. The main attributes of interest are `.type`, `.value` and `.traceback`.

Note that `pytest.raises` will match the exception type or any subclasses (like the standard `except` statement). If you want to check if a block of code is raising an exact exception type, you need to check that explicitly:

```
def test_foo_not_implemented():
    def foo():
        raise NotImplementedError

    with pytest.raises(RuntimeError) as excinfo:
        foo()
    assert excinfo.type is RuntimeError
```

The *pytest.raises()* call will succeed, even though the function raises `NotImplementedError`, because `NotImplementedError` is a subclass of `RuntimeError`; however the following `assert` statement will catch the problem.

#### Matching exception messages

You can pass a `match` keyword parameter to the context-manager to test that a regular expression matches on the string representation of an exception (similar to the `TestCase.assertRaisesRegex` method from `unittest`):

```
import pytest


def myfunc():
    raise ValueError("Exception 123 raised")


def test_match():
    with pytest.raises(ValueError, match=r".* 123 .*"):
        myfunc()
```

Notes:

- The `match` parameter is matched with the `re.search()` function, so in the above example `match='123'` would have worked as well.
- The `match` parameter also matches against PEP-678 `__notes__`.

### Assertions about expected exception groups

When expecting a `BaseExceptionGroup` or `ExceptionGroup` you can use *pytest.RaisesGroup*:

```
def test_exception_in_group():
    with pytest.RaisesGroup(ValueError):
        raise ExceptionGroup("group msg", [ValueError("value msg")])
    with pytest.RaisesGroup(ValueError, TypeError):
        raise ExceptionGroup("msg", [ValueError("foo"), TypeError("bar")])
```

It accepts a `match` parameter, that checks against the group message, and a `check` parameter that takes an arbitrary callable which it passes the group to, and only succeeds if the callable returns `True`.

```
def test_raisesgroup_match_and_check():
    with pytest.RaisesGroup(BaseException, match="my group msg"):
        raise BaseExceptionGroup("my group msg", [KeyboardInterrupt()])
    with pytest.RaisesGroup(
        Exception, check=lambda eg: isinstance(eg.__cause__, ValueError)
    ):
        raise ExceptionGroup("", [TypeError()]) from ValueError()
```

It is strict about structure and unwrapped exceptions, unlike except*, so you might want to set the `flatten_subgroups` and/or `allow_unwrapped` parameters.

```
def test_structure():
    with pytest.RaisesGroup(pytest.RaisesGroup(ValueError)):
        raise ExceptionGroup("", (ExceptionGroup("", (ValueError(),)),))
    with pytest.RaisesGroup(ValueError, flatten_subgroups=True):
        raise ExceptionGroup("1st group", [ExceptionGroup("2nd group",␣
↪[ValueError()])])
    with pytest.RaisesGroup(ValueError, allow_unwrapped=True):
        raise ValueError
```

To specify more details about the contained exception you can use *pytest.RaisesExc*

```
def test_raises_exc():
    with pytest.RaisesGroup(pytest.RaisesExc(ValueError, match="foo")):
        raise ExceptionGroup("", (ValueError("foo")))
```

They both supply a method *pytest.RaisesGroup.matches()* *pytest.RaisesExc.matches()* if you want to do matching outside of using it as a context manager. This can be helpful when checking `.__context__` or `.__cause__`.

```
def test_matches():
    exc = ValueError()
    exc_group = ExceptionGroup("", [exc])
    if RaisesGroup(ValueError).matches(exc_group):
        ...
    # helpful error is available in `.fail_reason` if it fails to match
    r = RaisesExc(ValueError)
    assert r.matches(e), r.fail_reason
```

Check the documentation on *pytest.RaisesGroup* and *pytest.RaisesExc* for more details and examples.

**ExceptionInfo.group_contains()**

**Warning**

This helper makes it easy to check for the presence of specific exceptions, but it is very bad for checking that the group does *not* contain *any other exceptions*. So this will pass:

```
class EXTREMELYBADERROR(BaseException):
    """This is a very bad error to miss"""


def test_for_value_error():
    with pytest.raises(ExceptionGroup) as excinfo:
        excs = [ValueError()]
        if very_unlucky():
            excs.append(EXTREMELYBADERROR())
        raise ExceptionGroup("", excs)
    # This passes regardless of if there's other exceptions.
    assert excinfo.group_contains(ValueError)
    # You can't simply list all exceptions you *don't* want to get here.
```

There is no good way of using *excinfo.group_contains()* to ensure you're not getting *any* other exceptions than the one you expected. You should instead use *pytest.RaisesGroup*, see *Assertions about expected exception groups*.

You can also use the *excinfo.group_contains()* method to test for exceptions returned as part of an ExceptionGroup:

```
def test_exception_in_group():
    with pytest.raises(ExceptionGroup) as excinfo:
        raise ExceptionGroup(
            "Group message",
            [
                RuntimeError("Exception 123 raised"),
            ],
        )
    assert excinfo.group_contains(RuntimeError, match=r".* 123 .*")
    assert not excinfo.group_contains(TypeError)
```

The optional `match` keyword parameter works the same way as for *pytest.raises()*.

By default `group_contains()` will recursively search for a matching exception at any level of nested `ExceptionGroup` instances. You can specify a `depth` keyword parameter if you only want to match an exception at a specific level; exceptions contained directly in the top `ExceptionGroup` would match `depth=1`.

```
def test_exception_in_group_at_given_depth():
    with pytest.raises(ExceptionGroup) as excinfo:
        raise ExceptionGroup(
            "Group message",
            [
                RuntimeError(),
                ExceptionGroup(
```

(continues on next page)

(continued from previous page)

```
                    "Nested group",
                    [
                        TypeError(),
                    ],
                ),
            ],
        )
    assert excinfo.group_contains(RuntimeError, depth=1)
    assert excinfo.group_contains(TypeError, depth=2)
    assert not excinfo.group_contains(RuntimeError, depth=2)
    assert not excinfo.group_contains(TypeError, depth=1)
```

### Alternate `pytest.raises` form (legacy)

There is an alternate form of *`pytest.raises()`* where you pass a function that will be executed, along with `*args` and `**kwargs`. *`pytest.raises()`* will then execute the function with those arguments and assert that the given exception is raised:

```
def func(x):
    if x <= 0:
        raise ValueError("x needs to be larger than zero")


pytest.raises(ValueError, func, x=-1)
```

This form was the original *`pytest.raises()`* API, developed before the `with` statement was added to the Python language. Nowadays, this form is rarely used, with the context-manager form (using `with`) being considered more readable.

### xfail mark and pytest.raises

It is also possible to specify a `raises` argument to *pytest.mark.xfail*, which checks that the test is failing in a more specific way than just having any exception raised:

```
def f():
    raise IndexError()


@pytest.mark.xfail(raises=IndexError)
def test_f():
    f()
```

This will only "xfail" if the test fails by raising `IndexError` or subclasses.

- Using *pytest.mark.xfail* with the `raises` parameter is probably better for something like documenting unfixed bugs (where the test describes what "should" happen) or bugs in dependencies.
- Using *`pytest.raises()`* is likely to be better for cases where you are testing exceptions your own code is deliberately raising, which is the majority of cases.

You can also use *`pytest.RaisesGroup`*:

```
def f():
    raise ExceptionGroup("", [IndexError()])
```

(continues on next page)

(continued from previous page)

```
@pytest.mark.xfail(raises=RaisesGroup(IndexError))
def test_f():
    f()
```

### 2.2.4 Assertions about expected warnings

You can check that code raises a particular warning using *pytest.warns*.

### 2.2.5 Making use of context-sensitive comparisons

`pytest` has rich support for providing context-sensitive information when it encounters comparisons. For example:

```
# content of test_assert2.py
def test_set_comparison():
    set1 = set("1308")
    set2 = set("8035")
    assert set1 == set2
```

if you run this module:

```
$ pytest test_assert2.py
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_assert2.py F                                                      [100%]

================================= FAILURES =================================
_____________________________ test_set_comparison ______________________________

    def test_set_comparison():
        set1 = set("1308")
        set2 = set("8035")
>       assert set1 == set2
E       AssertionError: assert {'0', '1', '3', '8'} == {'0', '3', '5', '8'}
E
E         Extra items in the left set:
E         '1'
E         Extra items in the right set:
E         '5'
E         Use -v to get more diff

test_assert2.py:4: AssertionError
========================== short test summary info ==========================
FAILED test_assert2.py::test_set_comparison - AssertionError: assert {'0'...
============================ 1 failed in 0.12s =============================
```

Special comparisons are done for a number of cases:

- comparing long strings: a context diff is shown

- comparing long sequences: first failing indices
- comparing dicts: different entries

In string context diffs, lines prefixed with - come from the left-hand side of `assert left == right`, while lines prefixed with + come from the right-hand side.

See the *reporting demo* for many more examples.

### 2.2.6 Defining your own explanation for failed assertions

It is possible to add your own detailed explanations by implementing the `pytest_assertrepr_compare` hook.

**pytest_assertrepr_compare**(*config*, *op*, *left*, *right*)

Return explanation for comparisons in failing assert expressions.

Return None for no custom explanation, otherwise return a list of strings. The strings will be joined by newlines but any newlines *in* a string will be escaped. Note that all but the first line will be indented slightly, the intention is for the first line to be a summary.

**Parameters**

- **config** – The pytest config object.
- **op** – The operator, e.g. `"=="`, `"!="`, `"not in"`.
- **left** – The left operand.
- **right** – The right operand.

#### Use in conftest plugins

Any conftest file can implement this hook. For a given item, only conftest files in the item's directory and its parent directories are consulted.

As an example consider adding the following hook in a *conftest.py* file which provides an alternative explanation for `Foo` objects:

```
# content of conftest.py
from test_foocompare import Foo


def pytest_assertrepr_compare(op, left, right):
    if isinstance(left, Foo) and isinstance(right, Foo) and op == "==":
        return [
            "Comparing Foo instances:",
            f"   vals: {left.val} != {right.val}",
        ]
```

now, given this test module:

```
# content of test_foocompare.py
class Foo:
    def __init__(self, val):
        self.val = val

    def __eq__(self, other):
        return self.val == other.val
```

(continues on next page)

(continued from previous page)

```
def test_compare():
    f1 = Foo(1)
    f2 = Foo(2)
    assert f1 == f2
```

you can run the test module and get the custom output defined in the conftest file:

```
$ pytest -q test_foocompare.py
F                                                                    [100%]
================================= FAILURES =================================
_________________________________ test_compare _________________________________

    def test_compare():
        f1 = Foo(1)
        f2 = Foo(2)
>       assert f1 == f2
E       assert Comparing Foo instances:
E            vals: 1 != 2

test_foocompare.py:12: AssertionError
========================= short test summary info ==========================
FAILED test_foocompare.py::test_compare - assert Comparing Foo instances:
1 failed in 0.12s
```

### 2.2.7 Returning non-None value in test functions

A *pytest.PytestReturnNotNoneWarning* is emitted when a test function returns a value other than `None`.

This helps prevent a common mistake made by beginners who assume that returning a `bool` (e.g., `True` or `False`) will determine whether a test passes or fails.

Example:

```
@pytest.mark.parametrize(
    ["a", "b", "result"],
    [
        [1, 2, 5],
        [2, 3, 8],
        [5, 3, 18],
    ],
)
def test_foo(a, b, result):
    return foo(a, b) == result  # Incorrect usage, do not do this.
```

Since pytest ignores return values, it might be surprising that the test will never fail based on the returned value.

The correct fix is to replace the `return` statement with an `assert`:

```
@pytest.mark.parametrize(
    ["a", "b", "result"],
    [
        [1, 2, 5],
        [2, 3, 8],
```

(continues on next page)

(continued from previous page)

```
        [5, 3, 18],
    ],
)
def test_foo(a, b, result):
    assert foo(a, b) == result
```

### 2.2.8 Assertion introspection details

Reporting details about a failing assertion is achieved by rewriting assert statements before they are run. Rewritten assert statements put introspection information into the assertion failure message. `pytest` only rewrites test modules directly discovered by its test collection process, so **asserts in supporting modules which are not themselves test modules will not be rewritten**.

You can manually enable assertion rewriting for an imported module by calling *register_assert_rewrite* before you import it (a good place to do that is in your root `conftest.py`).

For further information, Benjamin Peterson wrote up Behind the scenes of pytest's new assertion rewriting.

#### Assertion rewriting caches files on disk

`pytest` will write back the rewritten modules to disk for caching. You can disable this behavior (for example to avoid leaving stale `.pyc` files around in projects that move files around a lot) by adding this to the top of your `conftest.py` file:

```
import sys

sys.dont_write_bytecode = True
```

Note that you still get the benefits of assertion introspection, the only change is that the `.pyc` files won't be cached on disk.

Additionally, rewriting will silently skip caching if it cannot write new `.pyc` files, e.g. in a read-only filesystem or a zipfile.

#### Disabling assert rewriting

`pytest` rewrites test modules on import by using an import hook to write new `pyc` files. Most of the time this works transparently. However, if you are working with the import machinery yourself, the import hook may interfere.

If this is the case you have two options:

- Disable rewriting for a specific module by adding the string `PYTEST_DONT_REWRITE` to its docstring.
- Disable rewriting for all modules by using *`--assert=plain`*.

## 2.3 How to use fixtures

**See also**

*About fixtures*

**See also**

*Fixtures reference*

### 2.3.1 "Requesting" fixtures

At a basic level, test functions request fixtures they require by declaring them as arguments.

When pytest goes to run a test, it looks at the parameters in that test function's signature, and then searches for fixtures that have the same names as those parameters. Once pytest finds them, it runs those fixtures, captures what they returned (if anything), and passes those objects into the test function as arguments.

#### Quick example

```
import pytest


class Fruit:
    def __init__(self, name):
        self.name = name
        self.cubed = False

    def cube(self):
        self.cubed = True


class FruitSalad:
    def __init__(self, *fruit_bowl):
        self.fruit = fruit_bowl
        self._cube_fruit()

    def _cube_fruit(self):
        for fruit in self.fruit:
            fruit.cube()


# Arrange
@pytest.fixture
def fruit_bowl():
    return [Fruit("apple"), Fruit("banana")]


def test_fruit_salad(fruit_bowl):
    # Act
    fruit_salad = FruitSalad(*fruit_bowl)

    # Assert
    assert all(fruit.cubed for fruit in fruit_salad.fruit)
```

In this example, `test_fruit_salad` "**requests**" `fruit_bowl` (i.e. `def test_fruit_salad(fruit_bowl):`), and when pytest sees this, it will execute the `fruit_bowl` fixture function and pass the object it returns into `test_fruit_salad` as the `fruit_bowl` argument.

Here's roughly what's happening if we were to do it by hand:

```
def fruit_bowl():
    return [Fruit("apple"), Fruit("banana")]


def test_fruit_salad(fruit_bowl):
    # Act
    fruit_salad = FruitSalad(*fruit_bowl)

    # Assert
    assert all(fruit.cubed for fruit in fruit_salad.fruit)


# Arrange
bowl = fruit_bowl()
test_fruit_salad(fruit_bowl=bowl)
```

### Fixtures can request other fixtures

One of pytest's greatest strengths is its extremely flexible fixture system. It allows us to boil down complex requirements for tests into more simple and organized functions, where we only need to have each one describe the things they are dependent on. We'll get more into this further down, but for now, here's a quick example to demonstrate how fixtures can use other fixtures:

```
# contents of test_append.py
import pytest


# Arrange
@pytest.fixture
def first_entry():
    return "a"


# Arrange
@pytest.fixture
def order(first_entry):
    return [first_entry]


def test_string(order):
    # Act
    order.append("b")

    # Assert
    assert order == ["a", "b"]
```

Notice that this is the same example from above, but very little changed. The fixtures in pytest **request** fixtures just like tests. All the same **requesting** rules apply to fixtures that do for tests. Here's how this example would work if we did it by hand:

```
def first_entry():
    return "a"
```

(continues on next page)

(continued from previous page)

```
def order(first_entry):
    return [first_entry]


def test_string(order):
    # Act
    order.append("b")

    # Assert
    assert order == ["a", "b"]


entry = first_entry()
the_list = order(first_entry=entry)
test_string(order=the_list)
```

### Fixtures are reusable

One of the things that makes pytest's fixture system so powerful, is that it gives us the ability to define a generic setup step that can be reused over and over, just like a normal function would be used. Two different tests can request the same fixture and have pytest give each test their own result from that fixture.

This is extremely useful for making sure tests aren't affected by each other. We can use this system to make sure each test gets its own fresh batch of data and is starting from a clean state so it can provide consistent, repeatable results.

Here's an example of how this can come in handy:

```
# contents of test_append.py
import pytest


# Arrange
@pytest.fixture
def first_entry():
    return "a"


# Arrange
@pytest.fixture
def order(first_entry):
    return [first_entry]


def test_string(order):
    # Act
    order.append("b")

    # Assert
    assert order == ["a", "b"]


def test_int(order):
```

(continues on next page)

(continued from previous page)

```
    # Act
    order.append(2)

    # Assert
    assert order == ["a", 2]
```

Each test here is being given its own copy of that `list` object, which means the `order` fixture is getting executed twice (the same is true for the `first_entry` fixture). If we were to do this by hand as well, it would look something like this:

```
def first_entry():
    return "a"


def order(first_entry):
    return [first_entry]


def test_string(order):
    # Act
    order.append("b")

    # Assert
    assert order == ["a", "b"]


def test_int(order):
    # Act
    order.append(2)

    # Assert
    assert order == ["a", 2]


entry = first_entry()
the_list = order(first_entry=entry)
test_string(order=the_list)

entry = first_entry()
the_list = order(first_entry=entry)
test_int(order=the_list)
```

### A test/fixture can request more than one fixture at a time

Tests and fixtures aren't limited to **requesting** a single fixture at a time. They can request as many as they like. Here's another quick example to demonstrate:

```
# contents of test_append.py
import pytest


# Arrange
@pytest.fixture
```

(continues on next page)

(continued from previous page)

```
def first_entry():
    return "a"


# Arrange
@pytest.fixture
def second_entry():
    return 2


# Arrange
@pytest.fixture
def order(first_entry, second_entry):
    return [first_entry, second_entry]


# Arrange
@pytest.fixture
def expected_list():
    return ["a", 2, 3.0]


def test_string(order, expected_list):
    # Act
    order.append(3.0)

    # Assert
    assert order == expected_list
```

### Fixtures can be requested more than once per test (return values are cached)

Fixtures can also be **requested** more than once during the same test, and pytest won't execute them again for that test. This means we can **request** fixtures in multiple fixtures that are dependent on them (and even again in the test itself) without those fixtures being executed more than once.

```
# contents of test_append.py
import pytest


# Arrange
@pytest.fixture
def first_entry():
    return "a"


# Arrange
@pytest.fixture
def order():
    return []


# Act
```

(continues on next page)

(continued from previous page)

```
@pytest.fixture
def append_first(order, first_entry):
    return order.append(first_entry)


def test_string_only(append_first, order, first_entry):
    # Assert
    assert order == [first_entry]
```

If a **requested** fixture was executed once for every time it was **requested** during a test, then this test would fail because both `append_first` and `test_string_only` would see `order` as an empty list (i.e. `[]`), but since the return value of `order` was cached (along with any side effects executing it may have had) after the first time it was called, both the test and `append_first` were referencing the same object, and the test saw the effect `append_first` had on that object.

### 2.3.2 Autouse fixtures (fixtures you don't have to request)

Sometimes you may want to have a fixture (or even several) that you know all your tests will depend on. "Autouse" fixtures are a convenient way to make all tests automatically **request** them. This can cut out a lot of redundant **requests**, and can even provide more advanced fixture usage (more on that further down).

We can make a fixture an autouse fixture by passing in `autouse=True` to the fixture's decorator. Here's a simple example for how they can be used:

```
# contents of test_append.py
import pytest


@pytest.fixture
def first_entry():
    return "a"


@pytest.fixture
def order(first_entry):
    return []


@pytest.fixture(autouse=True)
def append_first(order, first_entry):
    return order.append(first_entry)


def test_string_only(order, first_entry):
    assert order == [first_entry]


def test_string_and_int(order, first_entry):
    order.append(2)
    assert order == [first_entry, 2]
```

In this example, the `append_first` fixture is an autouse fixture. Because it happens automatically, both tests are affected by it, even though neither test **requested** it. That doesn't mean they *can't* be **requested** though; just that it isn't *necessary*.

### 2.3.3 Scope: sharing fixtures across classes, modules, packages or session

Fixtures requiring network access depend on connectivity and are usually time-expensive to create. Extending the previous example, we can add a `scope="module"` parameter to the *`@pytest.fixture`* invocation to cause a `smtp_connection` fixture function, responsible to create a connection to a preexisting SMTP server, to only be invoked once per test *module* (the default is to invoke once per test *function*). Multiple test functions in a test module will thus each receive the same `smtp_connection` fixture instance, thus saving time. Possible values for `scope` are: `function`, `class`, `module`, `package` or `session`.

The next example puts the fixture function into a separate `conftest.py` file so that tests from multiple test modules in the directory can access the fixture function:

```
# content of conftest.py
import smtplib

import pytest


@pytest.fixture(scope="module")
def smtp_connection():
    return smtplib.SMTP("smtp.gmail.com", 587, timeout=5)
```

```
# content of test_module.py


def test_ehlo(smtp_connection):
    response, msg = smtp_connection.ehlo()
    assert response == 250
    assert b"smtp.gmail.com" in msg
    assert 0  # for demo purposes


def test_noop(smtp_connection):
    response, msg = smtp_connection.noop()
    assert response == 250
    assert 0  # for demo purposes
```

Here, the `test_ehlo` needs the `smtp_connection` fixture value. pytest will discover and call the *`@pytest.fixture`* marked `smtp_connection` fixture function. Running the test looks like this:

```
$ pytest test_module.py
========================== test session starts ==========================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 2 items

test_module.py FF                                                  [100%]

================================ FAILURES ================================
_________________________________ test_ehlo _________________________________

smtp_connection = <smtplib.SMTP object at 0xdeadbeef0001>

    def test_ehlo(smtp_connection):
```

(continues on next page)

(continued from previous page)

```
        response, msg = smtp_connection.ehlo()
        assert response == 250
        assert b"smtp.gmail.com" in msg
>       assert 0  # for demo purposes
        ^^^^^^^^
E       assert 0

test_module.py:7: AssertionError
_____________________________________ test_noop _____________________________________

smtp_connection = <smtplib.SMTP object at 0xdeadbeef0001>

    def test_noop(smtp_connection):
        response, msg = smtp_connection.noop()
        assert response == 250
>       assert 0  # for demo purposes
        ^^^^^^^^
E       assert 0

test_module.py:13: AssertionError
========================= short test summary info ==========================
FAILED test_module.py::test_ehlo - assert 0
FAILED test_module.py::test_noop - assert 0
============================ 2 failed in 0.12s =============================
```

You see the two `assert 0` failing and more importantly you can also see that the **exact same** `smtp_connection` object was passed into the two test functions because pytest shows the incoming argument values in the traceback. As a result, the two test functions using `smtp_connection` run as quick as a single one because they reuse the same instance.

If you decide that you rather want to have a session-scoped `smtp_connection` instance, you can simply declare it:

```
@pytest.fixture(scope="session")
def smtp_connection():
    # the returned fixture value will be shared for
    # all tests requesting it
    ...
```

### Fixture scopes

Fixtures are created when first requested by a test, and are destroyed based on their `scope`:

- `function`: the default scope, the fixture is destroyed at the end of the test.
- `class`: the fixture is destroyed during teardown of the last test in the class.
- `module`: the fixture is destroyed during teardown of the last test in the module.
- `package`: the fixture is destroyed during teardown of the last test in the package where the fixture is defined, including sub-packages and sub-directories within it.
- `session`: the fixture is destroyed at the end of the test session.

**Note**

Pytest only caches one instance of a fixture at a time, which means that when using a parametrized fixture, pytest may invoke a fixture more than once in the given scope.

#### Dynamic scope

Added in version 5.2.

In some cases, you might want to change the scope of the fixture without changing the code. To do that, pass a callable to `scope`. The callable must return a string with a valid scope and will be executed only once - during the fixture definition. It will be called with two keyword arguments - `fixture_name` as a string and `config` with a configuration object.

This can be especially useful when dealing with fixtures that need time for setup, like spawning a docker container. You can use the command-line argument to control the scope of the spawned containers for different environments. See the example below.

```
def determine_scope(fixture_name, config):
    if config.getoption("--keep-containers", None):
        return "session"
    return "function"


@pytest.fixture(scope=determine_scope)
def docker_container():
    yield spawn_container()
```

### 2.3.4 Teardown/Cleanup (AKA Fixture finalization)

When we run our tests, we'll want to make sure they clean up after themselves so they don't mess with any other tests (and also so that we don't leave behind a mountain of test data to bloat the system). Fixtures in pytest offer a very useful teardown system, which allows us to define the specific steps necessary for each fixture to clean up after itself.

This system can be leveraged in two ways.

#### 1. `yield` fixtures (recommended)

"Yield" fixtures `yield` instead of `return`. With these fixtures, we can run some code and pass an object back to the requesting fixture/test, just like with the other fixtures. The only differences are:

1. `return` is swapped out for `yield`.
2. Any teardown code for that fixture is placed *after* the `yield`.

Once pytest figures out a linear order for the fixtures, it will run each one up until it returns or yields, and then move on to the next fixture in the list to do the same thing.

Once the test is finished, pytest will go back down the list of fixtures, but in the *reverse order*, taking each one that yielded, and running the code inside it that was *after* the `yield` statement.

As a simple example, consider this basic email module:

```
# content of emaillib.py
class MailAdminClient:
    def create_user(self):
        return MailUser()

    def delete_user(self, user):
```

(continues on next page)

(continued from previous page)

```
        # do some cleanup
        pass


class MailUser:
    def __init__(self):
        self.inbox = []

    def send_email(self, email, other):
        other.inbox.append(email)

    def clear_mailbox(self):
        self.inbox.clear()


class Email:
    def __init__(self, subject, body):
        self.subject = subject
        self.body = body
```

Let's say we want to test sending email from one user to another. We'll have to first make each user, then send the email from one user to the other, and finally assert that the other user received that message in their inbox. If we want to clean up after the test runs, we'll likely have to make sure the other user's mailbox is emptied before deleting that user, otherwise the system may complain.

Here's what that might look like:

```
# content of test_emaillib.py
from emaillib import Email, MailAdminClient

import pytest


@pytest.fixture
def mail_admin():
    return MailAdminClient()


@pytest.fixture
def sending_user(mail_admin):
    user = mail_admin.create_user()
    yield user
    mail_admin.delete_user(user)


@pytest.fixture
def receiving_user(mail_admin):
    user = mail_admin.create_user()
    yield user
    user.clear_mailbox()
    mail_admin.delete_user(user)
```

(continues on next page)

(continued from previous page)

```
def test_email_received(sending_user, receiving_user):
    email = Email(subject="Hey!", body="How's it going?")
    sending_user.send_email(email, receiving_user)
    assert email in receiving_user.inbox
```

Because `receiving_user` is the last fixture to run during setup, it's the first to run during teardown.

There is a risk that even having the order right on the teardown side of things doesn't guarantee a safe cleanup. That's covered in a bit more detail in *Safe teardowns*.

```
$ pytest -q test_emaillib.py
.                                                                    [100%]
1 passed in 0.12s
```

### Handling errors for yield fixture

If a yield fixture raises an exception before yielding, pytest won't try to run the teardown code after that yield fixture's `yield` statement. But, for every fixture that has already run successfully for that test, pytest will still attempt to tear them down as it normally would.

### 2. Adding finalizers directly

While yield fixtures are considered to be the cleaner and more straightforward option, there is another choice, and that is to add "finalizer" functions directly to the test's *request-context* object. It brings a similar result as yield fixtures, but requires a bit more verbosity.

In order to use this approach, we have to request the *request-context* object (just like we would request another fixture) in the fixture we need to add teardown code for, and then pass a callable, containing that teardown code, to its `addfinalizer` method.

We have to be careful though, because pytest will run that finalizer once it's been added, even if that fixture raises an exception after adding the finalizer. So to make sure we don't run the finalizer code when we wouldn't need to, we would only add the finalizer once the fixture would have done something that we'd need to teardown.

Here's how the previous example would look using the `addfinalizer` method:

```
# content of test_emaillib.py
from emaillib import Email, MailAdminClient

import pytest


@pytest.fixture
def mail_admin():
    return MailAdminClient()


@pytest.fixture
def sending_user(mail_admin):
    user = mail_admin.create_user()
    yield user
    mail_admin.delete_user(user)
```

(continues on next page)

(continued from previous page)

```
@pytest.fixture
def receiving_user(mail_admin, request):
    user = mail_admin.create_user()

    def delete_user():
        mail_admin.delete_user(user)

    request.addfinalizer(delete_user)
    return user


@pytest.fixture
def email(sending_user, receiving_user, request):
    _email = Email(subject="Hey!", body="How's it going?")
    sending_user.send_email(_email, receiving_user)

    def empty_mailbox():
        receiving_user.clear_mailbox()

    request.addfinalizer(empty_mailbox)
    return _email


def test_email_received(receiving_user, email):
    assert email in receiving_user.inbox
```

It's a bit longer than yield fixtures and a bit more complex, but it does offer some nuances for when you're in a pinch.

```
$ pytest -q test_emaillib.py
.                                                                    [100%]
1 passed in 0.12s
```

### Note on finalizer order

Finalizers are executed in a first-in-last-out order. For yield fixtures, the first teardown code to run is from the right-most fixture, i.e. the last test parameter.

```
# content of test_finalizers.py
import pytest


def test_bar(fix_w_yield1, fix_w_yield2):
    print("test_bar")


@pytest.fixture
def fix_w_yield1():
    yield
    print("after_yield_1")


@pytest.fixture
```

(continues on next page)

(continued from previous page)

```
def fix_w_yield2():
    yield
    print("after_yield_2")
```

```
$ pytest -s test_finalizers.py
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_finalizers.py test_bar
.after_yield_2
after_yield_1


============================ 1 passed in 0.12s =============================
```

For finalizers, the first fixture to run is last call to `request.addfinalizer`.

```
# content of test_finalizers.py
from functools import partial
import pytest


@pytest.fixture
def fix_w_finalizers(request):
    request.addfinalizer(partial(print, "finalizer_2"))
    request.addfinalizer(partial(print, "finalizer_1"))


def test_bar(fix_w_finalizers):
    print("test_bar")
```

```
$ pytest -s test_finalizers.py
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_finalizers.py test_bar
.finalizer_1
finalizer_2


============================ 1 passed in 0.12s =============================
```

This is so because yield fixtures use `addfinalizer` behind the scenes: when the fixture executes, `addfinalizer` registers a function that resumes the generator, which in turn calls the teardown code.

### 2.3.5 Safe teardowns

The fixture system of pytest is *very* powerful, but it's still being run by a computer, so it isn't able to figure out how to safely teardown everything we throw at it. If we aren't careful, an error in the wrong spot might leave stuff from our tests behind, and that can cause further issues pretty quickly.

For example, consider the following tests (based off of the mail example from above):

```
# content of test_emaillib.py
from emaillib import Email, MailAdminClient

import pytest


@pytest.fixture
def setup():
    mail_admin = MailAdminClient()
    sending_user = mail_admin.create_user()
    receiving_user = mail_admin.create_user()
    email = Email(subject="Hey!", body="How's it going?")
    sending_user.send_email(email, receiving_user)
    yield receiving_user, email
    receiving_user.clear_mailbox()
    mail_admin.delete_user(sending_user)
    mail_admin.delete_user(receiving_user)


def test_email_received(setup):
    receiving_user, email = setup
    assert email in receiving_user.inbox
```

This version is a lot more compact, but it's also harder to read, doesn't have a very descriptive fixture name, and none of the fixtures can be reused easily.

There's also a more serious issue, which is that if any of those steps in the setup raise an exception, none of the teardown code will run.

One option might be to go with the `addfinalizer` method instead of yield fixtures, but that might get pretty complex and difficult to maintain (and it wouldn't be compact anymore).

```
$ pytest -q test_emaillib.py
.                                                                    [100%]
1 passed in 0.12s
```

#### Safe fixture structure

The safest and simplest fixture structure requires limiting fixtures to only making one state-changing action each, and then bundling them together with their teardown code, as *the email examples above* showed.

The chance that a state-changing operation can fail but still modify state is negligible, as most of these operations tend to be transaction-based (at least at the level of testing where state could be left behind). So if we make sure that any successful state-changing action gets torn down by moving it to a separate fixture function and separating it from other, potentially failing state-changing actions, then our tests will stand the best chance at leaving the test environment the way they found it.

For an example, let's say we have a website with a login page, and we have access to an admin API where we can generate users. For our test, we want to:

1. Create a user through that admin API
2. Launch a browser using Selenium
3. Go to the login page of our site
4. Log in as the user we created
5. Assert that their name is in the header of the landing page

We wouldn't want to leave that user in the system, nor would we want to leave that browser session running, so we'll want to make sure the fixtures that create those things clean up after themselves.

Here's what that might look like:

> **Note**
>
> For this example, certain fixtures (i.e. `base_url` and `admin_credentials`) are implied to exist elsewhere. So for now, let's assume they exist, and we're just not looking at them.

```
from uuid import uuid4
from urllib.parse import urljoin

from selenium.webdriver import Chrome
import pytest

from src.utils.pages import LoginPage, LandingPage
from src.utils import AdminApiClient
from src.utils.data_types import User


@pytest.fixture
def admin_client(base_url, admin_credentials):
    return AdminApiClient(base_url, **admin_credentials)


@pytest.fixture
def user(admin_client):
    _user = User(name="Susan", username=f"testuser-{uuid4()}", password="P4$$word")
    admin_client.create_user(_user)
    yield _user
    admin_client.delete_user(_user)


@pytest.fixture
def driver():
    _driver = Chrome()
    yield _driver
    _driver.quit()


@pytest.fixture
def login(driver, base_url, user):
    driver.get(urljoin(base_url, "/login"))
    page = LoginPage(driver)
```

(continues on next page)

(continued from previous page)

```
    page.login(user)


@pytest.fixture
def landing_page(driver, login):
    return LandingPage(driver)


def test_name_on_landing_page_after_login(landing_page, user):
    assert landing_page.header == f"Welcome, {user.name}!"
```

The way the dependencies are laid out means it's unclear if the `user` fixture would execute before the `driver` fixture. But that's ok, because those are atomic operations, and so it doesn't matter which one runs first because the sequence of events for the test is still linearizable. But what *does* matter is that, no matter which one runs first, if the one raises an exception while the other would not have, neither will have left anything behind. If `driver` executes before `user`, and `user` raises an exception, the driver will still quit, and the user was never made. And if `driver` was the one to raise the exception, then the driver would never have been started and the user would never have been made.

### 2.3.6 Running multiple `assert` statements safely

Sometimes you may want to run multiple asserts after doing all that setup, which makes sense as, in more complex systems, a single action can kick off multiple behaviors. pytest has a convenient way of handling this and it combines a bunch of what we've gone over so far.

All that's needed is stepping up to a larger scope, then having the **act** step defined as an autouse fixture, and finally, making sure all the fixtures are targeting that higher level scope.

Let's pull *an example from above*, and tweak it a bit. Let's say that in addition to checking for a welcome message in the header, we also want to check for a sign out button, and a link to the user's profile.

Let's take a look at how we can structure that so we can run multiple asserts without having to repeat all those steps again.

> **Note**
>
> For this example, certain fixtures (i.e. `base_url` and `admin_credentials`) are implied to exist elsewhere. So for now, let's assume they exist, and we're just not looking at them.

```
# contents of tests/end_to_end/test_login.py
from uuid import uuid4
from urllib.parse import urljoin

from selenium.webdriver import Chrome
import pytest

from src.utils.pages import LoginPage, LandingPage
from src.utils import AdminApiClient
from src.utils.data_types import User


@pytest.fixture(scope="class")
def admin_client(base_url, admin_credentials):
    return AdminApiClient(base_url, **admin_credentials)
```

(continues on next page)

(continued from previous page)

```
@pytest.fixture(scope="class")
def user(admin_client):
    _user = User(name="Susan", username=f"testuser-{uuid4()}", password="P4$$word")
    admin_client.create_user(_user)
    yield _user
    admin_client.delete_user(_user)


@pytest.fixture(scope="class")
def driver():
    _driver = Chrome()
    yield _driver
    _driver.quit()


@pytest.fixture(scope="class")
def landing_page(driver, login):
    return LandingPage(driver)


class TestLandingPageSuccess:
    @pytest.fixture(scope="class", autouse=True)
    @classmethod
    def login(cls, driver, base_url, user):
        driver.get(urljoin(base_url, "/login"))
        page = LoginPage(driver)
        page.login(user)

    def test_name_in_header(self, landing_page, user):
        assert landing_page.header == f"Welcome, {user.name}!"

    def test_sign_out_button(self, landing_page):
        assert landing_page.sign_out_button.is_displayed()

    def test_profile_link(self, landing_page, user):
        profile_href = urljoin(base_url, f"/profile?id={user.profile_id}")
        assert landing_page.profile_link.get_attribute("href") == profile_href
```

Notice that the methods are only referencing `self` in the signature as a formality. No state is tied to the actual test class as it might be in the `unittest.TestCase` framework. Everything is managed by the pytest fixture system.

Each method only has to request the fixtures that it actually needs without worrying about order. This is because the **act** fixture is an autouse fixture, and it made sure all the other fixtures executed before it. There's no more changes of state that need to take place, so the tests are free to make as many non-state-changing queries as they want without risking stepping on the toes of the other tests.

The `login` fixture is defined inside the class as well, because not every one of the other tests in the module will be expecting a successful login, and the **act** may need to be handled a little differently for another test class. For example, if we wanted to write another test scenario around submitting bad credentials, we could handle it by adding something like this to the test file:

```
class TestLandingPageBadCredentials:
    @pytest.fixture(scope="class")
    @classmethod
    def faux_user(cls, user):
        _user = deepcopy(user)
        _user.password = "badpass"
        return _user

    def test_raises_bad_credentials_exception(self, login_page, faux_user):
        with pytest.raises(BadCredentialsException):
            login_page.login(faux_user)
```

### 2.3.7 Fixtures can introspect the requesting test context

Fixture functions can accept the *request* object to introspect the "requesting" test function, class or module context. Further extending the previous `smtp_connection` fixture example, let's read an optional server URL from the test module which uses our fixture:

```
# content of conftest.py
import smtplib

import pytest


@pytest.fixture(scope="module")
def smtp_connection(request):
    server = getattr(request.module, "smtpserver", "smtp.gmail.com")
    smtp_connection = smtplib.SMTP(server, 587, timeout=5)
    yield smtp_connection
    print(f"finalizing {smtp_connection} ({server})")
    smtp_connection.close()
```

We use the `request.module` attribute to optionally obtain an `smtpserver` attribute from the test module. If we just execute again, nothing much has changed:

```
$ pytest -s -q --tb=no test_module.py
FFfinalizing <smtplib.SMTP object at 0xdeadbeef0002> (smtp.gmail.com)

========================= short test summary info ==========================
FAILED test_module.py::test_ehlo - assert 0
FAILED test_module.py::test_noop - assert 0
2 failed in 0.12s
```

Let's quickly create another test module that actually sets the server URL in its module namespace:

```
# content of test_anothersmtp.py

smtpserver = "mail.python.org"  # will be read by smtp fixture


def test_showhelo(smtp_connection):
    assert 0, smtp_connection.helo()
```

Running it:

```
$ pytest -qq --tb=short test_anothersmtp.py
F                                                                    [100%]
================================= FAILURES =================================
_______________________________ test_showhelo ________________________________
test_anothersmtp.py:6: in test_showhelo
    assert 0, smtp_connection.helo()
E   AssertionError: (250, b'mail.python.org')
E   assert 0
------------------------- Captured stdout teardown -------------------------
finalizing <smtplib.SMTP object at 0xdeadbeef0003> (mail.python.org)
========================= short test summary info ==========================
FAILED test_anothersmtp.py::test_showhelo - AssertionError: (250, b'mail....
```

voila! The `smtp_connection` fixture function picked up our mail server name from the module namespace.

### 2.3.8 Using markers to pass data to fixtures

Using the *request* object, a fixture can also access markers which are applied to a test function. This can be useful to pass data into a fixture from a test:

```
import pytest


@pytest.fixture
def fixt(request):
    marker = request.node.get_closest_marker("fixt_data")
    if marker is None:
        # Handle missing marker in some way...
        data = None
    else:
        data = marker.args[0]

    # Do something with the data
    return data


@pytest.mark.fixt_data(42)
def test_fixt(fixt):
    assert fixt == 42
```

### 2.3.9 Factories as fixtures

The "factory as fixture" pattern can help in situations where the result of a fixture is needed multiple times in a single test. Instead of returning data directly, the fixture instead returns a function which generates the data. This function can then be called multiple times in the test.

Factories can have parameters as needed:

```
@pytest.fixture
def make_customer_record():
    def _make_customer_record(name):
        return {"name": name, "orders": []}
```

(continues on next page)

(continued from previous page)

```
    return _make_customer_record


def test_customer_records(make_customer_record):
    customer_1 = make_customer_record("Lisa")
    customer_2 = make_customer_record("Mike")
    customer_3 = make_customer_record("Meredith")
```

If the data created by the factory requires managing, the fixture can take care of that:

```
@pytest.fixture
def make_customer_record():
    created_records = []

    def _make_customer_record(name):
        record = models.Customer(name=name, orders=[])
        created_records.append(record)
        return record

    yield _make_customer_record

    for record in created_records:
        record.destroy()


def test_customer_records(make_customer_record):
    customer_1 = make_customer_record("Lisa")
    customer_2 = make_customer_record("Mike")
    customer_3 = make_customer_record("Meredith")
```

### 2.3.10 Parametrizing fixtures

Fixture functions can be parametrized in which case they will be called multiple times, each time executing the set of dependent tests, i.e. the tests that depend on this fixture. Test functions usually do not need to be aware of their re-running. Fixture parametrization helps to write exhaustive functional tests for components which themselves can be configured in multiple ways.

Extending the previous example, we can flag the fixture to create two `smtp_connection` fixture instances which will cause all tests using the fixture to run twice. The fixture function gets access to each parameter through the special *`request`* object:

```
# content of conftest.py
import smtplib

import pytest


@pytest.fixture(scope="module", params=["smtp.gmail.com", "mail.python.org"])
def smtp_connection(request):
    smtp_connection = smtplib.SMTP(request.param, 587, timeout=5)
    yield smtp_connection
    print(f"finalizing {smtp_connection}")
    smtp_connection.close()
```

The main change is the declaration of `params` with *`@pytest.fixture`*, a list of values for each of which the fixture function will execute and can access a value via `request.param`. No test function code needs to change. So let's just do another run:

```
$ pytest -q test_module.py
FFFF                                                                  [100%]
================================= FAILURES =================================
____________________________ test_ehlo[smtp.gmail.com] _____________________________

smtp_connection = <smtplib.SMTP object at 0xdeadbeef0004>

    def test_ehlo(smtp_connection):
        response, msg = smtp_connection.ehlo()
        assert response == 250
        assert b"smtp.gmail.com" in msg
>       assert 0  # for demo purposes
        ^^^^^^^^
E       assert 0

test_module.py:7: AssertionError
____________________________ test_noop[smtp.gmail.com] _____________________________

smtp_connection = <smtplib.SMTP object at 0xdeadbeef0004>

    def test_noop(smtp_connection):
        response, msg = smtp_connection.noop()
        assert response == 250
>       assert 0  # for demo purposes
        ^^^^^^^^
E       assert 0

test_module.py:13: AssertionError
____________________________ test_ehlo[mail.python.org] ____________________________

smtp_connection = <smtplib.SMTP object at 0xdeadbeef0005>

    def test_ehlo(smtp_connection):
        response, msg = smtp_connection.ehlo()
        assert response == 250
>       assert b"smtp.gmail.com" in msg
E       AssertionError: assert b'smtp.gmail.com' in b'mail.python.org\nPIPELINING\
↪nSIZE 51200000\nETRN\nSTARTTLS\nAUTH DIGEST-MD5 NTLM CRAM-MD5\nENHANCEDSTATUSCODES\
↪n8BITMIME\nDSN\nSMTPUTF8\nCHUNKING'

test_module.py:6: AssertionError
-------------------------- Captured stdout setup ---------------------------
finalizing <smtplib.SMTP object at 0xdeadbeef0004>
____________________________ test_noop[mail.python.org] ____________________________

smtp_connection = <smtplib.SMTP object at 0xdeadbeef0005>

    def test_noop(smtp_connection):
        response, msg = smtp_connection.noop()
```

(continues on next page)

(continued from previous page)

```
        assert response == 250
>       assert 0  # for demo purposes
        ^^^^^^^^
E       assert 0

test_module.py:13: AssertionError
------------------------- Captured stdout teardown -------------------------
finalizing <smtplib.SMTP object at 0xdeadbeef0005>
========================= short test summary info ==========================
FAILED test_module.py::test_ehlo[smtp.gmail.com] - assert 0
FAILED test_module.py::test_noop[smtp.gmail.com] - assert 0
FAILED test_module.py::test_ehlo[mail.python.org] - AssertionError: asser...
FAILED test_module.py::test_noop[mail.python.org] - assert 0
4 failed in 0.12s
```

We see that our two test functions each ran twice, against the different `smtp_connection` instances. Note also, that with the `mail.python.org` connection the second test fails in `test_ehlo` because a different server string is expected than what arrived.

pytest will build a string that is the test ID for each fixture value in a parametrized fixture, e.g. `test_ehlo[smtp.gmail.com]` and `test_ehlo[mail.python.org]` in the above examples. These IDs can be used with *-k* to select specific cases to run, and they will also identify the specific case when one is failing. Running pytest with *--collect-only* will show the generated IDs.

Numbers, strings, booleans and `None` will have their usual string representation used in the test ID. For other objects, pytest will make a string based on the argument name. It is possible to customise the string used in a test ID for a certain fixture value by using the `ids` keyword argument:

```python
# content of test_ids.py
import pytest


@pytest.fixture(params=[0, 1], ids=["spam", "ham"])
def a(request):
    return request.param


def test_a(a):
    pass


def idfn(fixture_value):
    if fixture_value == 0:
        return "eggs"
    else:
        return None


@pytest.fixture(params=[0, 1], ids=idfn)
def b(request):
    return request.param
```

(continues on next page)

(continued from previous page)

```
def test_b(b):
    pass
```

The above shows how `ids` can be either a list of strings to use or a function which will be called with the fixture value and then has to return a string to use. In the latter case if the function returns `None` then pytest's auto-generated ID will be used.

Running the above tests results in the following test IDs being used:

```
$ pytest --collect-only
=========================== test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 12 items

<Dir fixtures.rst-236>
  <Module test_anothersmtp.py>
    <Function test_showhelo[smtp.gmail.com]>
    <Function test_showhelo[mail.python.org]>
  <Module test_emaillib.py>
    <Function test_email_received>
  <Module test_finalizers.py>
    <Function test_bar>
  <Module test_ids.py>
    <Function test_a[spam]>
    <Function test_a[ham]>
    <Function test_b[eggs]>
    <Function test_b[1]>
  <Module test_module.py>
    <Function test_ehlo[smtp.gmail.com]>
    <Function test_noop[smtp.gmail.com]>
    <Function test_ehlo[mail.python.org]>
    <Function test_noop[mail.python.org]>

======================== 12 tests collected in 0.12s =======================
```

### 2.3.11 Using marks with parametrized fixtures

*`pytest.param()`* can be used to apply marks in values sets of parametrized fixtures in the same way that they can be used with *@pytest.mark.parametrize*.

Example:

```
# content of test_fixture_marks.py
import pytest


@pytest.fixture(params=[0, 1, pytest.param(2, marks=pytest.mark.skip)])
def data_set(request):
    return request.param


def test_data(data_set):
```

(continues on next page)

(continued from previous page)

```
    pass
```

Running this test will *skip* the invocation of `data_set` with value `2`:

```
$ pytest test_fixture_marks.py -v
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y -- $PYTHON_PREFIX/bin/
↪python
cachedir: .pytest_cache
rootdir: /home/sweet/project
collecting ... collected 3 items

test_fixture_marks.py::test_data[0] PASSED                           [ 33%]
test_fixture_marks.py::test_data[1] PASSED                           [ 66%]
test_fixture_marks.py::test_data[2] SKIPPED (unconditional skip)     [100%]

======================== 2 passed, 1 skipped in 0.12s ========================
```

### 2.3.12 Modularity: using fixtures from a fixture function

In addition to using fixtures in test functions, fixture functions can use other fixtures themselves. This contributes to a modular design of your fixtures and allows reuse of framework-specific fixtures across many projects. As a simple example, we can extend the previous example and instantiate an object `app` where we stick the already defined `smtp_connection` resource into it:

```
# content of test_appsetup.py

import pytest


class App:
    def __init__(self, smtp_connection):
        self.smtp_connection = smtp_connection


@pytest.fixture(scope="module")
def app(smtp_connection):
    return App(smtp_connection)


def test_smtp_connection_exists(app):
    assert app.smtp_connection
```

Here we declare an `app` fixture which receives the previously defined `smtp_connection` fixture and instantiates an `App` object with it. Let's run it:

```
$ pytest -v test_appsetup.py
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y -- $PYTHON_PREFIX/bin/
↪python
cachedir: .pytest_cache
rootdir: /home/sweet/project
```

(continues on next page)

(continued from previous page)

```
collecting ... collected 2 items

test_appsetup.py::test_smtp_connection_exists[smtp.gmail.com] PASSED [ 50%]
test_appsetup.py::test_smtp_connection_exists[mail.python.org] PASSED [100%]

============================ 2 passed in 0.12s =============================
```

Due to the parametrization of `smtp_connection`, the test will run twice with two different `App` instances and respective smtp servers. There is no need for the `app` fixture to be aware of the `smtp_connection` parametrization because pytest will fully analyse the fixture dependency graph.

Note that the `app` fixture has a scope of `module` and uses a module-scoped `smtp_connection` fixture. The example would still work if `smtp_connection` was cached on a `session` scope: it is fine for fixtures to use "broader" scoped fixtures but not the other way round: A session-scoped fixture could not use a module-scoped one in a meaningful way.

### 2.3.13 Automatic grouping of tests by fixture instances

pytest minimizes the number of active fixtures during test runs. If you have a parametrized fixture, then all the tests using it will first execute with one instance and then finalizers are called before the next fixture instance is created. Among other things, this eases testing of applications which create and use global state.

The following example uses two parametrized fixtures, one of which is scoped on a per-module basis, and all the functions perform `print` calls to show the setup/teardown flow:

```python
# content of test_module.py
import pytest


@pytest.fixture(scope="module", params=["mod1", "mod2"])
def modarg(request):
    param = request.param
    print("  SETUP modarg", param)
    yield param
    print("  TEARDOWN modarg", param)


@pytest.fixture(scope="function", params=[1, 2])
def otherarg(request):
    param = request.param
    print("  SETUP otherarg", param)
    yield param
    print("  TEARDOWN otherarg", param)


def test_0(otherarg):
    print("  RUN test0 with otherarg", otherarg)


def test_1(modarg):
    print("  RUN test1 with modarg", modarg)


def test_2(otherarg, modarg):
    print(f"  RUN test2 with otherarg {otherarg} and modarg {modarg}")
```

Let's run the tests in verbose mode and with looking at the print-output:

```
$ pytest -v -s test_module.py
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y -- $PYTHON_PREFIX/bin/
↪python
cachedir: .pytest_cache
rootdir: /home/sweet/project
collecting ... collected 8 items

test_module.py::test_0[1]   SETUP otherarg 1
  RUN test0 with otherarg 1
PASSED  TEARDOWN otherarg 1

test_module.py::test_0[2]   SETUP otherarg 2
  RUN test0 with otherarg 2
PASSED  TEARDOWN otherarg 2

test_module.py::test_1[mod1]   SETUP modarg mod1
  RUN test1 with modarg mod1
PASSED
test_module.py::test_2[mod1-1]   SETUP otherarg 1
  RUN test2 with otherarg 1 and modarg mod1
PASSED  TEARDOWN otherarg 1

test_module.py::test_2[mod1-2]   SETUP otherarg 2
  RUN test2 with otherarg 2 and modarg mod1
PASSED  TEARDOWN otherarg 2

test_module.py::test_1[mod2]   TEARDOWN modarg mod1
  SETUP modarg mod2
  RUN test1 with modarg mod2
PASSED
test_module.py::test_2[mod2-1]   SETUP otherarg 1
  RUN test2 with otherarg 1 and modarg mod2
PASSED  TEARDOWN otherarg 1

test_module.py::test_2[mod2-2]   SETUP otherarg 2
  RUN test2 with otherarg 2 and modarg mod2
PASSED  TEARDOWN otherarg 2
  TEARDOWN modarg mod2


============================ 8 passed in 0.12s =============================
```

You can see that the parametrized module-scoped `modarg` resource caused an ordering of test execution that led to the fewest possible "active" resources. The finalizer for the `mod1` parametrized resource was executed before the `mod2` resource was setup.

In particular notice that test_0 is completely independent and finishes first. Then test_1 is executed with `mod1`, then test_2 with `mod1`, then test_1 with `mod2` and finally test_2 with `mod2`.

The `otherarg` parametrized resource (having function scope) was set up before and torn down after every test that used it.

### 2.3.14 Use fixtures in classes and modules with `usefixtures`

Sometimes test functions do not directly need access to a fixture object. For example, tests may require to operate with an empty directory as the current working directory but otherwise do not care for the concrete directory. Here is how you can use the standard `tempfile` and pytest fixtures to achieve it. We separate the creation of the fixture into a `conftest.py` file:

```
# content of conftest.py

import os
import tempfile

import pytest


@pytest.fixture
def cleandir():
    with tempfile.TemporaryDirectory() as newpath:
        old_cwd = os.getcwd()
        os.chdir(newpath)
        yield
        os.chdir(old_cwd)
```

and declare its use in a test module via a `usefixtures` marker:

```
# content of test_setenv.py
import os

import pytest


@pytest.mark.usefixtures("cleandir")
class TestDirectoryInit:
    def test_cwd_starts_empty(self):
        assert os.listdir(os.getcwd()) == []
        with open("myfile", "w", encoding="utf-8") as f:
            f.write("hello")

    def test_cwd_again_starts_empty(self):
        assert os.listdir(os.getcwd()) == []
```

Due to the `usefixtures` marker, the `cleandir` fixture will be required for the execution of each test method, just as if you specified a "cleandir" function argument to each of them. Let's run it to verify our fixture is activated and the tests pass:

```
$ pytest -q
..                                                                   [100%]
2 passed in 0.12s
```

You can specify multiple fixtures like this:

```
@pytest.mark.usefixtures("cleandir", "anotherfixture")
def test(): ...
```

and you may specify fixture usage at the test module level using *pytestmark*:

```
pytestmark = pytest.mark.usefixtures("cleandir")
```

It is also possible to put fixtures required by all tests in your project into a configuration file:

```
# content of pytest.toml
[pytest]
usefixtures = ["cleandir"]
```

**Warning**

`@pytest.mark.usefixtures` cannot be used on **fixture functions**. For example, this is an error:

```
@pytest.mark.usefixtures("my_other_fixture")
@pytest.fixture
def my_fixture_that_sadly_wont_use_my_other_fixture(): ...
```

### 2.3.15 Overriding fixtures on various levels

In a relatively large test suite, you may want to *override* a fixture, to augment or change its behavior inside of certain test modules or directories.

#### Override a fixture on a directory (conftest) level

Given the tests file structure is:

```
tests/
    conftest.py
        # content of tests/conftest.py
        import pytest

        @pytest.fixture
        def username():
            return 'username'

    test_something.py
        # content of tests/test_something.py
        def test_username(username):
            assert username == 'username'

    subdir/
        conftest.py
            # content of tests/subdir/conftest.py
            import pytest

            @pytest.fixture
            def username(username):
                return 'overridden-' + username

        test_something_else.py
            # content of tests/subdir/test_something_else.py
            def test_username(username):
                assert username == 'overridden-username'
```

As you can see, a fixture with the same name can be overridden for a certain test directory level. Note that the `base` or `super` fixture can be accessed from the `overriding` fixture easily - used in the example above.

### Override a fixture on a test module level

Given the tests file structure is:

```
tests/
    conftest.py
        # content of tests/conftest.py
        import pytest

        @pytest.fixture
        def username():
            return 'username'

    test_something.py
        # content of tests/test_something.py
        import pytest

        @pytest.fixture
        def username(username):
            return 'overridden-' + username

        def test_username(username):
            assert username == 'overridden-username'

    test_something_else.py
        # content of tests/test_something_else.py
        import pytest

        @pytest.fixture
        def username(username):
            return 'overridden-else-' + username

        def test_username(username):
            assert username == 'overridden-else-username'
```

In the example above, a fixture with the same name can be overridden for a certain test module.

### Override a fixture with direct test parametrization

Given the tests file structure is:

```
tests/
    conftest.py
        # content of tests/conftest.py
        import pytest

        @pytest.fixture
        def username():
            return 'username'

        @pytest.fixture
```

(continues on next page)

(continued from previous page)

```
        def other_username(username):
            return 'other-' + username

    test_something.py
        # content of tests/test_something.py
        import pytest

        @pytest.mark.parametrize('username', ['directly-overridden-username'])
        def test_username(username):
            assert username == 'directly-overridden-username'

        @pytest.mark.parametrize('username', ['directly-overridden-username-other'])
        def test_username_other(other_username):
            assert other_username == 'other-directly-overridden-username-other'
```

In the example above, a fixture value is overridden by the test parameter value. Note that the value of the fixture can be overridden this way even if the test doesn't use it directly (doesn't mention it in the function prototype).

### Override a parametrized fixture with non-parametrized one and vice versa

Given the tests file structure is:

```
tests/
    conftest.py
        # content of tests/conftest.py
        import pytest

        @pytest.fixture(params=['one', 'two', 'three'])
        def parametrized_username(request):
            return request.param

        @pytest.fixture
        def non_parametrized_username(request):
            return 'username'

    test_something.py
        # content of tests/test_something.py
        import pytest

        @pytest.fixture
        def parametrized_username():
            return 'overridden-username'

        @pytest.fixture(params=['one', 'two', 'three'])
        def non_parametrized_username(request):
            return request.param

        def test_username(parametrized_username):
            assert parametrized_username == 'overridden-username'

        def test_parametrized_username(non_parametrized_username):
            assert non_parametrized_username in ['one', 'two', 'three']
```

(continues on next page)

(continued from previous page)

```
test_something_else.py
    # content of tests/test_something_else.py
    def test_username(parametrized_username):
        assert parametrized_username in ['one', 'two', 'three']

    def test_username(non_parametrized_username):
        assert non_parametrized_username == 'username'
```

In the example above, a parametrized fixture is overridden with a non-parametrized version, and a non-parametrized fixture is overridden with a parametrized version for certain test module. The same applies for the test directory level obviously.

### 2.3.16 Using fixtures from other projects

Usually projects that provide pytest support will use *entry points*, so just installing those projects into an environment will make those fixtures available for use.

In case you want to use fixtures from a project that does not use entry points, you can define `pytest_plugins` in your top `conftest.py` file to register that module as a plugin.

Suppose you have some fixtures in `mylibrary.fixtures` and you want to reuse them into your `app/tests` directory.

All you need to do is to define `pytest_plugins` in `app/tests/conftest.py` pointing to that module.

```
pytest_plugins = "mylibrary.fixtures"
```

This effectively registers `mylibrary.fixtures` as a plugin, making all its fixtures and hooks available to tests in `app/tests`.

**Note**

Sometimes users will *import* fixtures from other projects for use, however this is not recommended: importing fixtures into a module will register them in pytest as *defined* in that module.

This has minor consequences, such as appearing multiple times in `pytest --help`, but it is not **recommended** because this behavior might change/stop working in future versions.

## 2.4 How to mark test functions with attributes

By using the `pytest.mark` helper you can easily set metadata on your test functions. You can find the full list of builtin markers in the *API Reference*. Or you can list all the markers, including builtin and custom, using the CLI - `pytest --markers`.

Here are some of the builtin markers:

- *usefixtures* - use fixtures on a test function or class
- *filterwarnings* - filter certain warnings of a test function
- *skip* - always skip a test function
- *skipif* - skip a test function if a certain condition is met
- *xfail* - produce an "expected failure" outcome if a certain condition is met
- *parametrize* - perform multiple calls to the same test function.

It's easy to create custom markers or to apply markers to whole test classes or modules. Those markers can be used by plugins, and also are commonly used to *select tests* on the command-line with the `-m` option.

See *Working with custom markers* for examples which also serve as documentation.

**Note**

Marks can only be applied to tests, having no effect on *fixtures*.

### 2.4.1 Registering marks

You can register custom marks in your configuration file like this:

```
[pytest]
markers = [
    "slow: marks tests as slow (deselect with '-m \"not slow\"')",
    "serial",
]
```

```
[pytest]
markers =
    slow: marks tests as slow (deselect with '-m "not slow"')
    serial
```

Note that everything past the `:` after the mark name is an optional description.

Alternatively, you can register new markers programmatically in a *pytest_configure* hook:

```
def pytest_configure(config):
    config.addinivalue_line(
        "markers", "env(name): mark test to run only on named environment"
    )
```

Registered marks appear in pytest's help text and do not emit warnings (see the next section). It is recommended that third-party plugins always *register their markers*.

### 2.4.2 Raising errors on unknown marks

Unregistered marks applied with the `@pytest.mark.name_of_the_mark` decorator will always emit a warning in order to avoid silently doing something surprising due to mistyped names. As described in the previous section, you can disable the warning for custom marks by registering them in your configuration file or using a custom `pytest_configure` hook.

When the `strict_markers` configuration option is set, any unknown marks applied with the `@pytest.mark.name_of_the_mark` decorator will trigger an error. You can enforce this validation in your project by setting `strict_markers` in your configuration:

```
[pytest]
addopts = ["--strict-markers"]
markers = [
    "slow: marks tests as slow (deselect with '-m \"not slow\"')",
    "serial",
]
```

```
[pytest]
strict_markers = true
markers =
    slow: marks tests as slow (deselect with '-m "not slow"')
    serial
```

## 2.5 How to parametrize fixtures and test functions

pytest enables test parametrization at several levels:

- *pytest.fixture()* allows one to *parametrize fixture functions*.
- *@pytest.mark.parametrize* allows one to define multiple sets of arguments and fixtures at the test function or class.
- *pytest_generate_tests* allows one to define custom parametrization schemes or extensions.

**Note**

See subtests for an alternative to parametrization.

### 2.5.1 `@pytest.mark.parametrize`: parametrizing test functions

The builtin *pytest.mark.parametrize* decorator enables parametrization of arguments for a test function. Here is a typical example of a test function that implements checking that a certain input leads to an expected output:

```
# content of test_expectation.py
import pytest


@pytest.mark.parametrize("test_input,expected", [("3+5", 8), ("2+4", 6), ("6*9", 42)])
def test_eval(test_input, expected):
    assert eval(test_input) == expected
```

Here, the `@parametrize` decorator defines three different `(test_input,expected)` tuples so that the `test_eval` function will run three times using them in turn:

```
$ pytest
========================== test session starts ==========================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 3 items

test_expectation.py ..F                                           [100%]

================================= FAILURES =================================
_____________________________ test_eval[6*9-42] _____________________________

test_input = '6*9', expected = 42

    @pytest.mark.parametrize("test_input,expected", [("3+5", 8), ("2+4", 6), ("6*9",␣
↪42)])
    def test_eval(test_input, expected):
```

(continues on next page)

(continued from previous page)

```
>       assert eval(test_input) == expected
E       AssertionError: assert 54 == 42
E        +  where 54 = eval('6*9')

test_expectation.py:6: AssertionError
========================== short test summary info ==========================
FAILED test_expectation.py::test_eval[6*9-42] - AssertionError: assert 54...
======================== 1 failed, 2 passed in 0.12s ========================
```

**Note**

Parameter values are passed as-is to tests (no copy whatsoever).

For example, if you pass a list or a dict as a parameter value, and the test case code mutates it, the mutations will be reflected in subsequent test case calls.

**Note**

pytest by default escapes any non-ascii characters used in unicode strings for the parametrization because it has several downsides. If however you would like to use unicode strings in parametrization and see them in the terminal as is (non-escaped), use this option in your configuration file:

```
[pytest]
disable_test_id_escaping_and_forfeit_all_rights_to_community_support = true
```

```
[pytest]
disable_test_id_escaping_and_forfeit_all_rights_to_community_support = true
```

Keep in mind however that this might cause unwanted side effects and even bugs depending on the OS used and plugins currently installed, so use it at your own risk.

As designed in this example, only one pair of input/output values fails the simple test function. And as usual with test function arguments, you can see the `input` and `output` values in the traceback.

Note that you could also use the parametrize marker on a class or a module (see *How to mark test functions with attributes*) which would invoke several functions with the argument sets, for instance:

```
import pytest


@pytest.mark.parametrize("n,expected", [(1, 2), (3, 4)])
class TestClass:
    def test_simple_case(self, n, expected):
        assert n + 1 == expected

    def test_weird_simple_case(self, n, expected):
        assert (n * 1) + 1 == expected
```

To parametrize all tests in a module, you can assign to the *pytestmark* global variable:

```
import pytest
```

(continues on next page)

(continued from previous page)

```
pytestmark = pytest.mark.parametrize("n,expected", [(1, 2), (3, 4)])


class TestClass:
    def test_simple_case(self, n, expected):
        assert n + 1 == expected

    def test_weird_simple_case(self, n, expected):
        assert (n * 1) + 1 == expected
```

It is also possible to mark individual test instances within parametrize, for example with the builtin `mark.xfail`:

```
# content of test_expectation.py
import pytest


@pytest.mark.parametrize(
    "test_input,expected",
    [("3+5", 8), ("2+4", 6), pytest.param("6*9", 42, marks=pytest.mark.xfail)],
)
def test_eval(test_input, expected):
    assert eval(test_input) == expected
```

Let's run this:

```
$ pytest
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 3 items

test_expectation.py ..x                                                [100%]

======================= 2 passed, 1 xfailed in 0.12s ========================
```

The one parameter set which caused a failure previously now shows up as an "xfailed" (expected to fail) test.

In case the values provided to `parametrize` result in an empty list - for example, if they're dynamically generated by some function - the behaviour of pytest is defined by the *empty_parameter_set_mark* option.

To get all combinations of multiple parametrized arguments you can stack `parametrize` decorators:

```
import pytest


@pytest.mark.parametrize("x", [0, 1])
@pytest.mark.parametrize("y", [2, 3])
def test_foo(x, y):
    pass
```

This will run the test with the arguments set to `x=0/y=2`, `x=1/y=2`, `x=0/y=3`, and `x=1/y=3` exhausting parameters in the order of the decorators.

### 2.5.2 Basic `pytest_generate_tests` example

Sometimes you may want to implement your own parametrization scheme or implement some dynamism for determining the parameters or scope of a fixture. For this, you can use the `pytest_generate_tests` hook which is called when collecting a test function. Through the passed in `metafunc` object you can inspect the requesting test context and, most importantly, you can call `metafunc.parametrize()` to cause parametrization.

For example, let's say we want to run a test taking string inputs which we want to set via a new `pytest` command line option. Let's first write a simple test accepting a `stringinput` fixture function argument:

```
# content of test_strings.py


def test_valid_string(stringinput):
    assert stringinput.isalpha()
```

Now we add a `conftest.py` file containing the addition of a command line option and the parametrization of our test function:

```
# content of conftest.py


def pytest_addoption(parser):
    parser.addoption(
        "--stringinput",
        action="append",
        default=[],
        help="list of stringinputs to pass to test functions",
    )


def pytest_generate_tests(metafunc):
    if "stringinput" in metafunc.fixturenames:
        metafunc.parametrize("stringinput", metafunc.config.getoption("stringinput"))
```

**Note**

The *pytest_generate_tests* hook can also be implemented directly in a test module or inside a test class; unlike other hooks, pytest will discover it there as well. Other hooks must live in a *conftest.py* or a plugin. See *Writing hook functions*.

If we now pass two stringinput values, our test will run twice:

```
$ pytest -q --stringinput="hello" --stringinput="world" test_strings.py
..                                                                   [100%]
2 passed in 0.12s
```

Let's also run with a stringinput that will lead to a failing test:

```
$ pytest -q --stringinput="!" test_strings.py
F                                                                    [100%]
================================ FAILURES ================================
_____________________________ test_valid_string[!] _____________________________
```

(continues on next page)

(continued from previous page)

```
stringinput = '!'

    def test_valid_string(stringinput):
>       assert stringinput.isalpha()
E       AssertionError: assert False
E        +  where False = <built-in method isalpha of str object at 0xdeadbeef0001>()
E        +    where <built-in method isalpha of str object at 0xdeadbeef0001> = '!'.
↪isalpha

test_strings.py:4: AssertionError
========================= short test summary info ==========================
FAILED test_strings.py::test_valid_string[!] - AssertionError: assert False
1 failed in 0.12s
```

As expected our test function fails.

If you don't specify a stringinput it will be skipped because `metafunc.parametrize()` will be called with an empty parameter list:

```
$ pytest -q -rs test_strings.py
s                                                                    [100%]
========================= short test summary info ==========================
SKIPPED [1] test_strings.py: got empty parameter set for (stringinput)
1 skipped in 0.12s
```

Note that when calling `metafunc.parametrize` multiple times with different parameter sets, all parameter names across those sets cannot be duplicated, otherwise an error will be raised.

### 2.5.3 More examples

For further examples, you might want to look at *more parametrization examples*.

## 2.6 How to use temporary directories and files in tests

### 2.6.1 The `tmp_path` fixture

You can use the `tmp_path` fixture which will provide a temporary directory unique to each test function.

`tmp_path` is a `pathlib.Path` object. Here is an example test usage:

```python
# content of test_tmp_path.py
CONTENT = "content"


def test_create_file(tmp_path):
    d = tmp_path / "sub"
    d.mkdir()
    p = d / "hello.txt"
    p.write_text(CONTENT, encoding="utf-8")
    assert p.read_text(encoding="utf-8") == CONTENT
    assert len(list(tmp_path.iterdir())) == 1
    assert 0
```

Running this would result in a passed test except for the last `assert 0` line which we use to look at values:

```
$ pytest test_tmp_path.py
============================= test session starts ==============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_tmp_path.py F                                                       [100%]

================================= FAILURES =================================
_________________________________ test_create_file _________________________________

tmp_path = PosixPath('PYTEST_TMPDIR/test_create_file0')

    def test_create_file(tmp_path):
        d = tmp_path / "sub"
        d.mkdir()
        p = d / "hello.txt"
        p.write_text(CONTENT, encoding="utf-8")
        assert p.read_text(encoding="utf-8") == CONTENT
        assert len(list(tmp_path.iterdir())) == 1
>       assert 0
E       assert 0

test_tmp_path.py:11: AssertionError
========================== short test summary info ==========================
FAILED test_tmp_path.py::test_create_file - assert 0
============================== 1 failed in 0.12s ===============================
```

By default, `pytest` retains the temporary directory for the last 3 `pytest` invocations. Concurrent invocations of the same test function are supported by configuring the base temporary directory to be unique for each concurrent run. See *temporary directory location and retention* for details.

### 2.6.2 The `tmp_path_factory` fixture

The `tmp_path_factory` is a session-scoped fixture which can be used to create arbitrary temporary directories from any other fixture or test.

For example, suppose your test suite needs a large image on disk, which is generated procedurally. Instead of computing the same image for each test that uses it into its own `tmp_path`, you can generate it once per-session to save time:

```
# contents of conftest.py
import pytest


@pytest.fixture(scope="session")
def image_file(tmp_path_factory):
    img = compute_expensive_image()
    fn = tmp_path_factory.mktemp("data") / "img.png"
    img.save(fn)
    return fn
```

(continues on next page)

(continued from previous page)

```
# contents of test_image.py
def test_histogram(image_file):
    img = load_image(image_file)
    # compute and test histogram
```

See *tmp_path_factory API* for details.

### 2.6.3 The `tmpdir` and `tmpdir_factory` fixtures

The `tmpdir` and `tmpdir_factory` fixtures are similar to `tmp_path` and `tmp_path_factory`, but use/return legacy py.path.local objects rather than standard `pathlib.Path` objects.

**Note**

These days, it is preferred to use `tmp_path` and `tmp_path_factory`.

In order to help modernize old code bases, one can run pytest with the legacypath plugin disabled:

```
pytest -p no:legacypath
```

This will trigger errors on tests using the legacy paths. It can also be permanently set as part of the *addopts* parameter in the config file.

See *tmpdir tmpdir_factory* API for details.

### 2.6.4 Temporary directory location and retention

The temporary directories, as returned by the *tmp_path* and (now deprecated) *tmpdir* fixtures, are automatically created under a base temporary directory, in a structure that depends on the *--basetemp* option:

- By default (when the *--basetemp* option is not set), the temporary directories will follow this template:

  ```
  {temproot}/pytest-of-{user}/pytest-{num}/{testname}/
  ```

  where:

  - `{temproot}` is the system temporary directory as determined by `tempfile.gettempdir()`. It can be overridden by the *PYTEST_DEBUG_TEMPROOT* environment variable.
  - `{user}` is the user name running the tests,
  - `{num}` is a number that is incremented with each test suite run
  - `{testname}` is a sanitized version of *the name of the current test*.

  The auto-incrementing `{num}` placeholder provides a basic retention feature and avoids that existing results of previous test runs are blindly removed. By default, the last 3 temporary directories are kept, but this behavior can be configured with *tmp_path_retention_count* and *tmp_path_retention_policy*.

- When the *--basetemp* option is used (e.g. `pytest --basetemp=mydir`), it will be used directly as base temporary directory:

  ```
  {basetemp}/{testname}/
  ```

  Note that there is no retention feature in this case: only the results of the most recent run will be kept.

> **Warning**
>
> The directory given to *--basetemp* will be cleared blindly before each test run, so make sure to use a directory for that purpose only.

When distributing tests on the local machine using `pytest-xdist`, care is taken to automatically configure a `basetemp` directory for the sub processes such that all temporary data lands below a single per-test run temporary directory.

## 2.7 How to monkeypatch/mock modules and environments

Sometimes tests need to invoke functionality which depends on global settings or which invokes code which cannot be easily tested such as network access. The `monkeypatch` fixture helps you to safely set/delete an attribute, dictionary item or environment variable, or to modify `sys.path` for importing.

The `monkeypatch` fixture provides these helper methods for safely patching and mocking functionality in tests:

- *monkeypatch.setattr(obj, name, value, raising=True)*
- *monkeypatch.delattr(obj, name, raising=True)*
- *monkeypatch.setitem(mapping, name, value)*
- *monkeypatch.delitem(obj, name, raising=True)*
- *monkeypatch.setenv(name, value, prepend=None)*
- *monkeypatch.delenv(name, raising=True)*
- *monkeypatch.syspath_prepend(path)*
- *monkeypatch.chdir(path)*
- *monkeypatch.context()*

All modifications will be undone after the requesting test function or fixture has finished. The `raising` parameter determines if a `KeyError` or `AttributeError` will be raised if the target of the set/deletion operation does not exist.

Consider the following scenarios:

1. Modifying the behavior of a function or the property of a class for a test e.g. there is an API call or database connection you will not make for a test but you know what the expected output should be. Use *monkeypatch.setattr* to patch the function or property with your desired testing behavior. This can include your own functions. Use *monkeypatch.delattr* to remove the function or property for the test.

2. Modifying the values of dictionaries e.g. you have a global configuration that you want to modify for certain test cases. Use *monkeypatch.setitem* to patch the dictionary for the test. *monkeypatch.delitem* can be used to remove items.

3. Modifying environment variables for a test e.g. to test program behavior if an environment variable is missing, or to set multiple values to a known variable. *monkeypatch.setenv* and *monkeypatch.delenv* can be used for these patches.

4. Use `monkeypatch.setenv("PATH", value, prepend=os.pathsep)` to modify `$PATH`, and *monkeypatch.chdir* to change the context of the current working directory during a test.

5. Use *monkeypatch.syspath_prepend* to modify `sys.path` which will also call `pkg_resources.fixup_namespace_packages` and `importlib.invalidate_caches()`.

6. Use *monkeypatch.context* to apply patches only in a specific scope, which can help control teardown of complex fixtures or patches to the stdlib.

See the monkeypatch blog post for some introduction material and a discussion of its motivation.

### 2.7.1 Monkeypatching functions

Consider a scenario where you are working with user directories. In the context of testing, you do not want your test to depend on the running user. `monkeypatch` can be used to patch functions dependent on the user to always return a specific value.

In this example, *`monkeypatch.setattr`* is used to patch `Path.home` so that the known testing path `Path("/abc")` is always used when the test is run. This removes any dependency on the running user for testing purposes. *`monkeypatch.setattr`* must be called before the function which will use the patched function is called. After the test function finishes the `Path.home` modification will be undone.

```
# contents of test_module.py with source code and the test
from pathlib import Path


def getssh():
    """Simple function to return expanded homedir ssh path."""
    return Path.home() / ".ssh"


def test_getssh(monkeypatch):
    # mocked return function to replace Path.home
    # always return '/abc'
    def mockreturn():
        return Path("/abc")

    # Application of the monkeypatch to replace Path.home
    # with the behavior of mockreturn defined above.
    monkeypatch.setattr(Path, "home", mockreturn)

    # Calling getssh() will use mockreturn in place of Path.home
    # for this test with the monkeypatch.
    x = getssh()
    assert x == Path("/abc/.ssh")
```

### 2.7.2 Monkeypatching returned objects: building mock classes

*`monkeypatch.setattr`* can be used in conjunction with classes to mock returned objects from functions instead of values. Imagine a simple function to take an API url and return the json response.

```
# contents of app.py, a simple API retrieval example
import requests


def get_json(url):
    """Takes a URL, and returns the JSON."""
    r = requests.get(url)
    return r.json()
```

We need to mock `r`, the returned response object for testing purposes. The mock of `r` needs a `.json()` method which returns a dictionary. This can be done in our test file by defining a class to represent `r`.

```
# contents of test_app.py, a simple test for our API retrieval
# import requests for the purposes of monkeypatching
```

(continues on next page)

(continued from previous page)

```
import requests

# our app.py that includes the get_json() function
# this is the previous code block example
import app


# custom class to be the mock return value
# will override the requests.Response returned from requests.get
class MockResponse:
    # mock json() method always returns a specific testing dictionary
    @staticmethod
    def json():
        return {"mock_key": "mock_response"}


def test_get_json(monkeypatch):
    # Any arguments may be passed and mock_get() will always return our
    # mocked object, which only has the .json() method.
    def mock_get(*args, **kwargs):
        return MockResponse()

    # apply the monkeypatch for requests.get to mock_get
    monkeypatch.setattr(requests, "get", mock_get)

    # app.get_json, which contains requests.get, uses the monkeypatch
    result = app.get_json("https://fakeurl")
    assert result["mock_key"] == "mock_response"
```

`monkeypatch` applies the mock for `requests.get` with our `mock_get` function. The `mock_get` function returns an instance of the `MockResponse` class, which has a `json()` method defined to return a known testing dictionary and does not require any outside API connection.

You can build the `MockResponse` class with the appropriate degree of complexity for the scenario you are testing. For instance, it could include an `ok` property that always returns `True`, or return different values from the `json()` mocked method based on input strings.

This mock can be shared across tests using a `fixture`:

```
# contents of test_app.py, a simple test for our API retrieval
import pytest
import requests

# app.py that includes the get_json() function
import app


# custom class to be the mock return value of requests.get()
class MockResponse:
    @staticmethod
    def json():
        return {"mock_key": "mock_response"}
```

(continues on next page)

(continued from previous page)

```
# monkeypatched requests.get moved to a fixture
@pytest.fixture
def mock_response(monkeypatch):
    """Requests.get() mocked to return {'mock_key':'mock_response'}."""

    def mock_get(*args, **kwargs):
        return MockResponse()

    monkeypatch.setattr(requests, "get", mock_get)


# notice our test uses the custom fixture instead of monkeypatch directly
def test_get_json(mock_response):
    result = app.get_json("https://fakeurl")
    assert result["mock_key"] == "mock_response"
```

Furthermore, if the mock was designed to be applied to all tests, the `fixture` could be moved to a `conftest.py` file and use the with `autouse=True` option.

### 2.7.3 Global patch example: preventing "requests" from remote operations

If you want to prevent the "requests" library from performing http requests in all your tests, you can do:

```
# contents of conftest.py
import pytest


@pytest.fixture(autouse=True)
def no_requests(monkeypatch):
    """Remove requests.sessions.Session.request for all tests."""
    monkeypatch.delattr("requests.sessions.Session.request")
```

This autouse fixture will be executed for each test function and it will delete the method `request.session.Session.request` so that any attempts within tests to create http requests will fail.

**Note**

Be advised that it is not recommended to patch builtin functions such as `open`, `compile`, etc., because it might break pytest's internals. If that's unavoidable, passing *`--tb=native`*, *`--assert=plain`* and *`--capture=no`* might help although there's no guarantee.

**Note**

Mind that patching `stdlib` functions and some third-party libraries used by pytest might break pytest itself. Prefer patching the reference that your code uses instead of patching the original object in the standard library. For example, if your module does `from os import getcwd`, patch `mymodule.getcwd` rather than `os.getcwd`.

For code that you control, a safer long-term pattern is to make dependencies explicit so they can be passed into the code under test instead of patched globally. When patching a stdlib object is unavoidable, use *`MonkeyPatch.context()`* to limit the patching to the block you want tested:

```
import functools


def test_partial(monkeypatch):
    with monkeypatch.context() as m:
        m.setattr(functools, "partial", 3)
        assert functools.partial == 3
```

See #3290 for details.

### 2.7.4 Monkeypatching environment variables

If you are working with environment variables you often need to safely change the values or delete them from the system for testing purposes. `monkeypatch` provides a mechanism to do this using the `setenv` and `delenv` method. Our example code to test:

```
# contents of our original code file e.g. code.py
import os


def get_os_user_lower():
    """Simple retrieval function.
    Returns lowercase USER or raises OSError."""
    username = os.getenv("USER")

    if username is None:
        raise OSError("USER environment is not set.")

    return username.lower()
```

There are two potential paths. First, the `USER` environment variable is set to a value. Second, the `USER` environment variable does not exist. Using `monkeypatch` both paths can be safely tested without impacting the running environment:

```
# contents of our test file e.g. test_code.py
import pytest


def test_upper_to_lower(monkeypatch):
    """Set the USER env var to assert the behavior."""
    monkeypatch.setenv("USER", "TestingUser")
    assert get_os_user_lower() == "testinguser"


def test_raise_exception(monkeypatch):
    """Remove the USER env var and assert OSError is raised."""
    monkeypatch.delenv("USER", raising=False)

    with pytest.raises(OSError):
        _ = get_os_user_lower()
```

This behavior can be moved into `fixture` structures and shared across tests:

```
# contents of our test file e.g. test_code.py
import pytest


@pytest.fixture
def mock_env_user(monkeypatch):
    monkeypatch.setenv("USER", "TestingUser")


@pytest.fixture
def mock_env_missing(monkeypatch):
    monkeypatch.delenv("USER", raising=False)


# notice the tests reference the fixtures for mocks
def test_upper_to_lower(mock_env_user):
    assert get_os_user_lower() == "testinguser"


def test_raise_exception(mock_env_missing):
    with pytest.raises(OSError):
        _ = get_os_user_lower()
```

### 2.7.5 Monkeypatching dictionaries

*monkeypatch.setitem* can be used to safely set the values of dictionaries to specific values during tests. Take this simplified connection string example:

```
# contents of app.py to generate a simple connection string
DEFAULT_CONFIG = {"user": "user1", "database": "db1"}


def create_connection_string(config=None):
    """Creates a connection string from input or defaults."""
    config = config or DEFAULT_CONFIG
    return f"User Id={config['user']}; Location={config['database']};"
```

For testing purposes we can patch the `DEFAULT_CONFIG` dictionary to specific values.

```
# contents of test_app.py
# app.py with the connection string function (prior code block)
import app


def test_connection(monkeypatch):
    # Patch the values of DEFAULT_CONFIG to specific
    # testing values only for this test.
    monkeypatch.setitem(app.DEFAULT_CONFIG, "user", "test_user")
    monkeypatch.setitem(app.DEFAULT_CONFIG, "database", "test_db")

    # expected result based on the mocks
    expected = "User Id=test_user; Location=test_db;"
```

(continues on next page)

(continued from previous page)

```
    # the test uses the monkeypatched dictionary settings
    result = app.create_connection_string()
    assert result == expected
```

You can use the *monkeypatch.delitem* to remove values.

```
# contents of test_app.py
import pytest

# app.py with the connection string function
import app


def test_missing_user(monkeypatch):
    # patch the DEFAULT_CONFIG to be missing the 'user' key
    monkeypatch.delitem(app.DEFAULT_CONFIG, "user", raising=False)

    # Key error expected because a config is not passed, and the
    # default is now missing the 'user' entry.
    with pytest.raises(KeyError):
        _ = app.create_connection_string()
```

The modularity of fixtures gives you the flexibility to define separate fixtures for each potential mock and reference them in the needed tests.

```
# contents of test_app.py
import pytest

# app.py with the connection string function
import app


# all of the mocks are moved into separated fixtures
@pytest.fixture
def mock_test_user(monkeypatch):
    """Set the DEFAULT_CONFIG user to test_user."""
    monkeypatch.setitem(app.DEFAULT_CONFIG, "user", "test_user")


@pytest.fixture
def mock_test_database(monkeypatch):
    """Set the DEFAULT_CONFIG database to test_db."""
    monkeypatch.setitem(app.DEFAULT_CONFIG, "database", "test_db")


@pytest.fixture
def mock_missing_default_user(monkeypatch):
    """Remove the user key from DEFAULT_CONFIG"""
    monkeypatch.delitem(app.DEFAULT_CONFIG, "user", raising=False)


# tests reference only the fixture mocks that are needed
```

(continues on next page)

(continued from previous page)

```
def test_connection(mock_test_user, mock_test_database):
    expected = "User Id=test_user; Location=test_db;"

    result = app.create_connection_string()
    assert result == expected


def test_missing_user(mock_missing_default_user):
    with pytest.raises(KeyError):
        _ = app.create_connection_string()
```

### 2.7.6 API Reference

Consult the docs for the *MonkeyPatch* class.

## 2.8 How to run doctests

By default, all files matching the `test*.txt` pattern will be run through the python standard `doctest` module. You can change the pattern by issuing:

```
pytest --doctest-glob="*.rst"
```

on the command line. *--doctest-glob* can be given multiple times in the command-line.

If you then have a text file like this:

```
# content of test_example.txt

hello this is a doctest
>>> x = 3
>>> x
3
```

then you can just invoke `pytest` directly:

```
$ pytest
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_example.txt .                                                    [100%]

============================= 1 passed in 0.12s =============================
```

By default, pytest will collect `test*.txt` files looking for doctest directives, but you can pass additional globs using the *--doctest-glob* option (multi-allowed).

In addition to text files, you can also execute doctests directly from docstrings of your classes and functions, including from test modules, using the *--doctest-modules* option:

```
# content of mymodule.py
def something():
```

(continues on next page)

(continued from previous page)

```
    """a doctest in a docstring
    >>> something()
    42
    """
    return 42
```

```
$ pytest --doctest-modules
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 2 items

mymodule.py .                                                         [ 50%]
test_example.txt .                                                    [100%]

============================ 2 passed in 0.12s =============================
```

You can make these changes permanent in your project by putting them into a configuration file like this:

```
# content of pytest.toml
[pytest]
addopts = ["--doctest-modules"]
```

### 2.8.1 Encoding

The default encoding is **UTF-8**, but you can specify the encoding that will be used for those doctest files using the *doctest_encoding* configuration option:

```
[pytest]
doctest_encoding = "latin1"
```

```
[pytest]
doctest_encoding = latin1
```

### 2.8.2 Using 'doctest' options

Python's standard `doctest` module provides some options to configure the strictness of doctest tests. In pytest, you can enable those flags using the configuration file.

For example, to make pytest ignore trailing whitespaces and ignore lengthy exception stack traces you can just write:

```
[pytest]
doctest_optionflags = ["NORMALIZE_WHITESPACE", "IGNORE_EXCEPTION_DETAIL"]
```

```
[pytest]
doctest_optionflags = NORMALIZE_WHITESPACE IGNORE_EXCEPTION_DETAIL
```

Alternatively, options can be enabled by an inline comment in the doc test itself:

```
>>> something_that_raises()  # doctest: +IGNORE_EXCEPTION_DETAIL
Traceback (most recent call last):
ValueError: ...
```

pytest also introduces new options:

- `ALLOW_UNICODE`: when enabled, the `u` prefix is stripped from unicode strings in expected doctest output. This allows doctests to run in Python 2 and Python 3 unchanged.
- `ALLOW_BYTES`: similarly, the `b` prefix is stripped from byte strings in expected doctest output.
- `NUMBER`: when enabled, floating-point numbers only need to match as far as the precision you have written in the expected doctest output. The numbers are compared using *`pytest.approx()`* with relative tolerance equal to the precision. For example, the following output would only need to match to 2 decimal places when comparing `3.14` to `pytest.approx(math.pi, rel=10**-2)`:

  ```
  >>> math.pi
  3.14
  ```

  If you wrote `3.1416` then the actual output would need to match to approximately 4 decimal places; and so on.

  This avoids false positives caused by limited floating-point precision, like this:

  ```
  Expected:
      0.233
  Got:
      0.23300000000000001
  ```

  `NUMBER` also supports lists of floating-point numbers – in fact, it matches floating-point numbers appearing anywhere in the output, even inside a string! This means that it may not be appropriate to enable globally in `doctest_optionflags` in your configuration file.

  Added in version 5.1.

### 2.8.3 Continue on failure

By default, pytest would report only the first failure for a given doctest. If you want to continue the test even when you have failures, do:

```
pytest --doctest-modules --doctest-continue-on-failure
```

### 2.8.4 Output format

You can change the diff output format on failure for your doctests by using one of the standard doctest module's format options (see `doctest.REPORT_UDIFF`, `doctest.REPORT_CDIFF`, `doctest.REPORT_NDIFF`, `doctest.REPORT_ONLY_FIRST_FAILURE`):

```
pytest --doctest-modules --doctest-report none
pytest --doctest-modules --doctest-report udiff
pytest --doctest-modules --doctest-report cdiff
pytest --doctest-modules --doctest-report ndiff
pytest --doctest-modules --doctest-report only_first_failure
```

### 2.8.5 pytest-specific features

Some features are provided to make writing doctests easier or with better integration with your existing test suite. Keep in mind however that by using those features you will make your doctests incompatible with the standard `doctests` module.

### Using fixtures

It is possible to use fixtures using the `getfixture` helper:

```
# content of example.rst
>>> tmp = getfixture('tmp_path')
>>> ...
>>>
```

Note that the fixture needs to be defined in a place visible by pytest, for example, a `conftest.py` file or plugin; normal python files containing docstrings are not normally scanned for fixtures unless explicitly configured by *python_files*.

Also, the *usefixtures* mark and fixtures marked as *autouse* are supported when executing text doctest files.

Python doctest modules are collected independently from Python test files. Fixture scope is not shared between the two.

Doctests do not support fixtures that depend on parametrization, because doctest collection does not perform the same test generation as normal test functions. This includes parametrized autouse fixtures. If you need to run doctests against multiple backends or configurations, consider moving those checks into normal test functions or a dedicated doctest plugin.

### 'doctest_namespace' fixture

The `doctest_namespace` fixture can be used to inject items into the namespace in which your doctests run. It is intended to be used within your own fixtures to provide the tests that use them with context.

`doctest_namespace` is a standard `dict` object into which you place the objects you want to appear in the doctest namespace:

```
# content of conftest.py
import pytest
import numpy


@pytest.fixture(autouse=True)
def add_np(doctest_namespace):
    doctest_namespace["np"] = numpy
```

which can then be used in your doctests directly:

```
# content of numpy.py
def arange():
    """
    >>> a = np.arange(10)
    >>> len(a)
    10
    """
```

Note that like the normal `conftest.py`, the fixtures are discovered in the directory tree conftest is in. Meaning that if you put your doctest with your source code, the relevant conftest.py needs to be in the same directory tree. Fixtures will not be discovered in a sibling directory tree!

### Skipping tests

For the same reasons one might want to skip normal tests, it is also possible to skip tests inside doctests.

To skip a single check inside a doctest you can use the standard `doctest.SKIP` directive:

```
def test_random(y):
    """
    >>> random.random()  # doctest: +SKIP
    0.156231223

    >>> 1 + 1
    2
    """
```

This will skip the first check, but not the second.

pytest also allows using the standard pytest functions *pytest.skip()* and *pytest.xfail()* inside doctests, which might be useful because you can then skip/xfail tests based on external conditions:

```
>>> import sys, pytest
>>> if sys.platform.startswith('win'):
...     pytest.skip('this doctest does not work on Windows')
...
>>> import fcntl
>>> ...
```

However using those functions is discouraged because it reduces the readability of the docstring.

**Note**

*pytest.skip()* and *pytest.xfail()* behave differently depending if the doctests are in a Python file (in docstrings) or a text file containing doctests intermingled with text:

- Python modules (docstrings): the functions only act in that specific docstring, letting the other docstrings in the same module execute as normal.
- Text files: the functions will skip/xfail the checks for the rest of the entire file.

### 2.8.6 Alternatives

While the built-in pytest support provides a good set of functionalities for using doctests, if you use them extensively you might be interested in those external packages which add many more features, and include pytest integration:

- pytest-doctestplus: provides advanced doctest support and enables the testing of reStructuredText (".rst") files.
- Sybil: provides a way to test examples in your documentation by parsing them from the documentation source and evaluating the parsed examples as part of your normal test run.

## 2.9 How to re-run failed tests and maintain state between test runs

### 2.9.1 Usage

The plugin provides two command line options to rerun failures from the last `pytest` invocation:

- *--lf, --last-failed* - to only re-run the failures.
- *--ff, --failed-first* - to run the failures first and then the rest of the tests.

For cleanup (usually not needed), a *--cache-clear* option allows to remove all cross-session cache contents ahead of a test run.

Other plugins may access the *config.cache* object to set/get **json encodable** values between `pytest` invocations.

> **Note**
>
> This plugin is enabled by default, but can be disabled if needed: see *Deactivating / unregistering a plugin by name* (the internal name for this plugin is `cacheprovider`).

### 2.9.2 Rerunning only failures or failures first

First, let's create 50 test invocations of which only 2 fail:

```
# content of test_50.py
import pytest


@pytest.mark.parametrize("i", range(50))
def test_num(i):
    if i in (17, 25):
        pytest.fail("bad luck")
```

If you run this for the first time you will see two failures:

```
$ pytest -q
.................F.......F........................                   [100%]
================================= FAILURES =================================
_________________________________ test_num[17] _________________________________

i = 17

    @pytest.mark.parametrize("i", range(50))
    def test_num(i):
        if i in (17, 25):
>           pytest.fail("bad luck")
E           Failed: bad luck

test_50.py:7: Failed
_________________________________ test_num[25] _________________________________

i = 25

    @pytest.mark.parametrize("i", range(50))
    def test_num(i):
        if i in (17, 25):
>           pytest.fail("bad luck")
E           Failed: bad luck

test_50.py:7: Failed
========================== short test summary info ==========================
FAILED test_50.py::test_num[17] - Failed: bad luck
FAILED test_50.py::test_num[25] - Failed: bad luck
2 failed, 48 passed in 0.12s
```

If you then run it with `--lf`:

```
$ pytest --lf
========================== test session starts ==========================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 2 items
run-last-failure: rerun previous 2 failures

test_50.py FF                                                      [100%]

================================ FAILURES ================================
_____________________________ test_num[17] _____________________________

i = 17

    @pytest.mark.parametrize("i", range(50))
    def test_num(i):
        if i in (17, 25):
>           pytest.fail("bad luck")
E           Failed: bad luck

test_50.py:7: Failed
_____________________________ test_num[25] _____________________________

i = 25

    @pytest.mark.parametrize("i", range(50))
    def test_num(i):
        if i in (17, 25):
>           pytest.fail("bad luck")
E           Failed: bad luck

test_50.py:7: Failed
======================== short test summary info ========================
FAILED test_50.py::test_num[17] - Failed: bad luck
FAILED test_50.py::test_num[25] - Failed: bad luck
========================== 2 failed in 0.12s ============================
```

You have run only the two failing tests from the last run, while the 48 passing tests have not been run ("deselected").

Now, if you run with the `--ff` option, all tests will be run but the first previous failures will be executed first (as can be seen from the series of `FF` and dots):

```
$ pytest --ff
========================== test session starts ==========================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 50 items
run-last-failure: rerun previous 2 failures first

test_50.py FF................................................      [100%]

================================ FAILURES ================================
_____________________________ test_num[17] _____________________________
```

(continues on next page)

(continued from previous page)

```
i = 17

    @pytest.mark.parametrize("i", range(50))
    def test_num(i):
        if i in (17, 25):
>           pytest.fail("bad luck")
E           Failed: bad luck

test_50.py:7: Failed
_____________________________________ test_num[25] _____________________________________

i = 25

    @pytest.mark.parametrize("i", range(50))
    def test_num(i):
        if i in (17, 25):
>           pytest.fail("bad luck")
E           Failed: bad luck

test_50.py:7: Failed
========================== short test summary info ==========================
FAILED test_50.py::test_num[17] - Failed: bad luck
FAILED test_50.py::test_num[25] - Failed: bad luck
======================== 2 failed, 48 passed in 0.12s ========================
```

New *--nf, --new-first* option: run new tests first followed by the rest of the tests, in both cases tests are also sorted by the file modified time, with more recent files coming first.

### 2.9.3 Behavior when no tests failed in the last run

The *--lfnf, --last-failed-no-failures* option governs the behavior of *--last-failed*. Determines whether to execute tests when there are no previously (known) failures or when no cached `lastfailed` data was found.

There are two options:

- `all`: when there are no known test failures, runs all tests (the full test suite). This is the default.
- `none`: when there are no known test failures, just emits a message stating this and exit successfully.

Example:

```
pytest --last-failed --last-failed-no-failures all    # runs the full test suite␣
↪(default behavior)
pytest --last-failed --last-failed-no-failures none   # runs no tests and exits␣
↪successfully
```

### 2.9.4 The new config.cache object

Plugins or conftest.py support code can get a cached value using the pytest `config` object. Here is a basic example plugin which implements a *fixture* which reuses previously created state across pytest invocations:

```
# content of test_caching.py
import pytest
```

(continues on next page)

(continued from previous page)

```
def expensive_computation():
    print("running expensive computation...")


@pytest.fixture
def mydata(pytestconfig):
    cache = getattr(pytestconfig, "cache", None)
    if cache is None:
        # pytestconfig not having the cache attribute means the
        # cache plugin is disabled.
        expensive_computation()
        return 42

    val = cache.get("example/value", None)
    if val is None:
        expensive_computation()
        val = 42
        cache.set("example/value", val)
    return val


def test_function(mydata):
    assert mydata == 23
```

If you run this command for the first time, you can see the print statement:

```
$ pytest -q
F                                                                    [100%]
================================ FAILURES ================================
_____________________________ test_function _____________________________

mydata = 42

    def test_function(mydata):
>       assert mydata == 23
E       assert 42 == 23

test_caching.py:26: AssertionError
-------------------------- Captured stdout setup --------------------------
running expensive computation...
========================= short test summary info ========================
FAILED test_caching.py::test_function - assert 42 == 23
1 failed in 0.12s
```

If you run it a second time, the value will be retrieved from the cache and nothing will be printed:

```
$ pytest -q
F                                                                    [100%]
================================ FAILURES ================================
_____________________________ test_function _____________________________
```

(continues on next page)

(continued from previous page)

```
mydata = 42

    def test_function(mydata):
>       assert mydata == 23
E       assert 42 == 23

test_caching.py:26: AssertionError
========================= short test summary info ==========================
FAILED test_caching.py::test_function - assert 42 == 23
1 failed in 0.12s
```

See the *config.cache fixture* for more details.

### 2.9.5 Inspecting Cache content

You can always peek at the content of the cache using the *--cache-show* command line option:

```
$ pytest --cache-show
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
cachedir: /home/sweet/project/.pytest_cache
--------------------------- cache values for '*' ---------------------------
cache/lastfailed contains:
  {'test_caching.py::test_function': True}
cache/nodeids contains:
  ['test_caching.py::test_function']
example/value contains:
  42

========================== no tests ran in 0.12s ============================
```

*--cache-show* takes an optional argument to specify a glob pattern for filtering:

```
$ pytest --cache-show example/*
============================ test session starts ============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
cachedir: /home/sweet/project/.pytest_cache
---------------------- cache values for 'example/*' ----------------------
example/value contains:
  42

========================== no tests ran in 0.12s ============================
```

### 2.9.6 Clearing Cache content

You can instruct pytest to clear all cache files and values by adding the *--cache-clear* option like this:

```
pytest --cache-clear
```

This is recommended for invocations from Continuous Integration servers where isolation and correctness is more important than speed.

### 2.9.7 Stepwise

As an alternative to *--lf -x*, especially for cases where you expect a large part of the test suite will fail, *--sw, --stepwise* allows you to fix them one at a time. The test suite will run until the first failure and then stop. At the next invocation, tests will continue from the last failing test and then run until the next failing test. You may use the *--stepwise-skip* option to ignore one failing test and stop the test execution on the second failing test instead. This is useful if you get stuck on a failing test and just want to ignore it until later. Providing `--stepwise-skip` will also enable `--stepwise` implicitly.

## 2.10 How to manage logging

pytest captures log messages of level `WARNING` or above automatically and displays them in their own section for each failed test in the same manner as captured stdout and stderr.

Running without options:

```
pytest
```

Shows failed tests like so:

```
----------------------- Captured stdlog call ----------------------
test_reporting.py    26 WARNING  text going to logger
----------------------- Captured stdout call ----------------------
text going to stdout
----------------------- Captured stderr call ----------------------
text going to stderr
==================== 2 failed in 0.02 seconds =====================
```

By default each captured log message shows the module, line number, log level and message.

If desired the log and date format can be specified to anything that the logging module supports by passing specific formatting options:

```
pytest --log-format="%(asctime)s %(levelname)s %(message)s" \
       --log-date-format="%Y-%m-%d %H:%M:%S"
```

Shows failed tests like so:

```
----------------------- Captured stdlog call ----------------------
2010-04-10 14:48:44 WARNING text going to logger
----------------------- Captured stdout call ----------------------
text going to stdout
----------------------- Captured stderr call ----------------------
text going to stderr
==================== 2 failed in 0.02 seconds =====================
```

These options can also be customized through a configuration file:

```
[pytest]
log_format = "%(asctime)s %(levelname)s %(message)s"
log_date_format = "%Y-%m-%d %H:%M:%S"
```

```
[pytest]
log_format = %(asctime)s %(levelname)s %(message)s
log_date_format = %Y-%m-%d %H:%M:%S
```

Specific loggers can be disabled via *--log-disable={logger_name}*. This argument can be passed multiple times:

```
pytest --log-disable=main --log-disable=testing
```

Further it is possible to disable reporting of captured content (stdout, stderr and logs) on failed tests completely with:

```
pytest --show-capture=no
```

### 2.10.1 caplog fixture

Inside tests it is possible to change the log level for the captured log messages. This is supported by the `caplog` fixture:

```
def test_foo(caplog):
    caplog.set_level(logging.INFO)
```

By default the level is set on the root logger, however as a convenience it is also possible to set the log level of any logger:

```
def test_foo(caplog):
    caplog.set_level(logging.CRITICAL, logger="root.baz")
```

The log levels set are restored automatically at the end of the test.

It is also possible to use a context manager to temporarily change the log level inside a `with` block:

```
def test_bar(caplog):
    with caplog.at_level(logging.INFO):
        pass
```

Again, by default the level of the root logger is affected but the level of any logger can be changed instead with:

```
def test_bar(caplog):
    with caplog.at_level(logging.CRITICAL, logger="root.baz"):
        pass
```

Lastly all the logs sent to the logger during the test run are made available on the fixture in the form of both the `logging.LogRecord` instances and the final log text. This is useful for when you want to assert on the contents of a message:

```
def test_baz(caplog):
    func_under_test()
    for record in caplog.records:
        assert record.levelname != "CRITICAL"
    assert "wally" not in caplog.text
```

For all the available attributes of the log records see the `logging.LogRecord` class.

You can also resort to `record_tuples` if all you want to do is to ensure, that certain messages have been logged under a given logger name with a given severity and message:

```
def test_foo(caplog):
    logging.getLogger().info("boo %s", "arg")
```

(continues on next page)

(continued from previous page)

```
    assert caplog.record_tuples == [("root", logging.INFO, "boo arg")]
```

You can call `caplog.clear()` to reset the captured log records in a test:

```
def test_something_with_clearing_records(caplog):
    some_method_that_creates_log_records()
    caplog.clear()
    your_test_method()
    assert ["Foo"] == [rec.message for rec in caplog.records]
```

The `caplog.records` attribute contains records from the current stage only, so inside the `setup` phase it contains only setup logs, same with the `call` and `teardown` phases.

To access logs from other stages, use the `caplog.get_records(when)` method. As an example, if you want to make sure that tests which use a certain fixture never log any warnings, you can inspect the records for the `setup` and `call` stages during teardown like so:

```
@pytest.fixture
def window(caplog):
    window = create_window()
    yield window
    for when in ("setup", "call"):
        messages = [
            x.message for x in caplog.get_records(when) if x.levelno == logging.
↪WARNING
        ]
        if messages:
            pytest.fail(f"warning messages encountered during testing: {messages}")
```

The full API is available at *pytest.LogCaptureFixture*.

**Warning**

The `caplog` fixture adds a handler to the root logger to capture logs. If the root logger is modified during a test, for example with `logging.config.dictConfig`, this handler may be removed and cause no logs to be captured. To avoid this, ensure that any root logger configuration only adds to the existing handlers.

### 2.10.2 Live Logs

By setting the *log_cli* configuration option to `true`, pytest will output logging records as they are emitted directly into the console.

You can specify the logging level for which log records with equal or higher level are printed to the console by passing *--log-cli-level*. This setting accepts the logging level names or numeric values as seen in logging's documentation.

Additionally, you can also specify *--log-cli-format* and *--log-cli-date-format* which mirror and default to *--log-format* and *--log-date-format* if not provided, but are applied only to the console logging handler.

All of the CLI log options can also be set in the configuration file. The option names are:

- *log_cli_level*
- *log_cli_format*
- *log_cli_date_format*

If you need to record the whole test suite logging calls to a file, you can pass *--log-file=/path/to/log/file*. This log file is opened in write mode by default, which means that it will be overwritten at each test session. If you'd like the file opened in append mode instead, then you can pass *--log-file-mode=a*. Note that relative paths for the log-file location, whether passed on the CLI or declared in a config file, are always resolved relative to the current working directory.

You can also specify the logging level for the log file by passing *--log-file-level*. This setting accepts the logging level names or numeric values as seen in logging's documentation.

Additionally, you can also specify *--log-file-format* and *--log-file-date-format* which are equal to `--log-format` and *--log-date-format* but are applied to the log file logging handler.

All of the log file options can also be set in the configuration file. The option names are:

- *log_file*
- *log_file_mode*
- *log_file_level*
- *log_file_format*
- *log_file_date_format*

You can call `set_log_path()` to customize the log_file path dynamically. This functionality is considered **experimental**. Note that `set_log_path()` respects the *log_file_mode* option.

### 2.10.3 Customizing Colors

Log levels are colored if colored terminal output is enabled. Changing from default colors or putting color on custom log levels is supported through `add_color_level()`. Example:

```
@pytest.hookimpl(trylast=True)
def pytest_configure(config):
    logging_plugin = config.pluginmanager.get_plugin("logging-plugin")

    # Change color on existing log level
    logging_plugin.log_cli_handler.formatter.add_color_level(logging.INFO, "cyan")

    # Add color to a custom log level (a custom log level `SPAM` is already set up)
    logging_plugin.log_cli_handler.formatter.add_color_level(logging.SPAM, "blue")
```

> **Warning**
>
> This feature and its API are considered **experimental** and might change between releases without a deprecation notice.

### 2.10.4 Release notes

This feature was introduced as a drop-in replacement for the pytest-catchlog plugin and they conflict with each other. The backward compatibility API with `pytest-capturelog` has been dropped when this feature was introduced, so if for that reason you still need `pytest-catchlog` you can disable the internal feature by adding to your configuration file:

```
[pytest]
addopts = ["-p", "no:logging"]
```

```
[pytest]
addopts = -p no:logging
```

### 2.10.5 Incompatible changes in pytest 3.4

This feature was introduced in `3.3` and some **incompatible changes** have been made in `3.4` after community feedback:

- Log levels are no longer changed unless explicitly requested by the *`log_level`* configuration or *`--log-level`* command-line options. This allows users to configure logger objects themselves. Setting *`log_level`* will set the level that is captured globally so if a specific test requires a lower level than this, use the `caplog.set_level()` functionality otherwise that test will be prone to failure.
- *Live Logs* is now disabled by default and can be enabled setting the *`log_cli`* configuration option to `true`. When enabled, the verbosity is increased so logging for each test is visible.
- *Live Logs* are now sent to `sys.stdout` and no longer require the *`-s`* command-line option to work.

If you want to partially restore the logging behavior of version `3.3`, you can add these options to your configuration file:

```
[pytest]
log_cli = true
log_level = "NOTSET"
```

```
[pytest]
log_cli = true
log_level = NOTSET
```

More details about the discussion that led to these changes can be read in #3013.

## 2.11 How to capture stdout/stderr output

Pytest intercepts stdout and stderr as configured by the *`--capture=`* command-line argument or by using fixtures. The `--capture=` flag configures reporting, whereas the fixtures offer more granular control and allow inspection of output during testing. The reports can be customized with the *`-r`* flag.

### 2.11.1 Default stdout/stderr/stdin capturing behaviour

During test execution any output sent to `stdout` and `stderr` is captured. If a test or a setup method fails its according captured output will usually be shown along with the failure traceback. (This behavior can be configured by the *`--show-capture`* command-line option).

In addition, `stdin` is set to a "null" object which will fail on attempts to read from it because it is rarely desired to wait for interactive input when running automated tests.

By default capturing is done by intercepting writes to low level file descriptors. This allows capturing output from simple print statements as well as output from a subprocess started by a test.

### 2.11.2 Setting capturing methods or disabling capturing

There are three ways in which `pytest` can perform capturing:

- `fd` (file descriptor) level capturing (default): All writes going to the operating system file descriptors 1 and 2 will be captured.
- `sys` level capturing: Only writes to Python files `sys.stdout` and `sys.stderr` will be captured. No capturing of writes to filedescriptors is performed.

- `tee-sys` capturing: Python writes to `sys.stdout` and `sys.stderr` will be captured, however the writes will also be passed-through to the actual `sys.stdout` and `sys.stderr`. This allows output to be 'live printed' and captured for plugin use, such as junitxml (new in pytest 5.4).

You can influence output capturing mechanisms from the command line:

```
pytest -s                  # disable all capturing
pytest --capture=sys       # replace sys.stdout/stderr with in-mem files
pytest --capture=fd        # also point filedescriptors 1 and 2 to temp file
pytest --capture=tee-sys   # combines 'sys' and '-s', capturing sys.stdout/stderr
                           # and passing it along to the actual sys.stdout/stderr
```

### 2.11.3 Using print statements for debugging

One primary benefit of the default capturing of stdout/stderr output is that you can use print statements for debugging:

```
# content of test_module.py


def setup_function(function):
    print("setting up", function)


def test_func1():
    assert True


def test_func2():
    assert False
```

and running this module will show you precisely the output of the failing function and hide the other one:

```
$ pytest
========================== test session starts ==========================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 2 items

test_module.py .F                                                  [100%]

================================ FAILURES ================================
_____________________________ test_func2 _____________________________

    def test_func2():
>       assert False
E       assert False

test_module.py:12: AssertionError
-------------------------- Captured stdout setup --------------------------
setting up <function test_func2 at 0xdeadbeef0001>
======================== short test summary info ========================
FAILED test_module.py::test_func2 - assert False
====================== 1 failed, 1 passed in 0.12s ======================
```

### 2.11.4 Accessing captured output from a test function

The *capsys*, *capteesys*, *capsysbinary*, *capfd*, and *capfdbinary* fixtures allow access to `stdout`/`stderr` output created during test execution.

Here is an example test function that performs some output related checks:

```
def test_myoutput(capsys):  # or use "capfd" for fd-level
    print("hello")
    sys.stderr.write("world\n")
    captured = capsys.readouterr()
    assert captured.out == "hello\n"
    assert captured.err == "world\n"
    print("next")
    captured = capsys.readouterr()
    assert captured.out == "next\n"
```

The `readouterr()` call snapshots the output so far - and capturing will be continued. After the test function finishes the original streams will be restored. Using *capsys* this way frees your test from having to care about setting/resetting output streams and also interacts well with pytest's own per-test capturing.

The return value of `readouterr()` is a `namedtuple` with two attributes, `out` and `err`.

If the code under test writes non-textual data (`bytes`), you can capture this using the *capsysbinary* fixture which instead returns `bytes` from the `readouterr` method.

If you want to capture at the file descriptor level you can use the *capfd* fixture which offers the exact same interface but allows to also capture output from libraries or subprocesses that directly write to operating system level output streams (FD1 and FD2). Similarly to *capsysbinary*, *capfdbinary* can be used to capture `bytes` at the file descriptor level.

To temporarily disable capture within a test, the capture fixtures have a `disabled()` method that can be used as a context manager, disabling capture inside the `with` block:

```
def test_disabling_capturing(capsys):
    print("this output is captured")
    with capsys.disabled():
        print("output not captured, going directly to sys.stdout")
    print("this output is also captured")
```

**Note**

When a capture fixture such as *capsys* or *capfd* is used, it takes precedence over the global capturing configuration set via command-line options such as `-s` or `--capture=no`.

This means that output produced within a test using a capture fixture will still be captured and available via `readouterr()`, even if global capturing is disabled.

## 2.12 How to capture warnings

Starting from version `3.1`, pytest now automatically catches warnings during test execution and displays them at the end of the session:

```
# content of test_show_warnings.py
import warnings
```

(continues on next page)

(continued from previous page)

```
def api_v1():
    warnings.warn(UserWarning("api v1, should use functions from v2"))
    return 1


def test_one():
    assert api_v1() == 1
```

Running pytest now produces this output:

```
$ pytest test_show_warnings.py
============================= test session starts ==============================
platform linux -- Python 3.x.y, pytest-9.x.y, pluggy-1.x.y
rootdir: /home/sweet/project
collected 1 item

test_show_warnings.py .                                                  [100%]

============================== warnings summary ===============================
test_show_warnings.py::test_one
  /home/sweet/project/test_show_warnings.py:5: UserWarning: api v1, should use␣
↪functions from v2
    warnings.warn(UserWarning("api v1, should use functions from v2"))

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 1 passed, 1 warning in 0.12s =========================
```

### 2.12.1 Controlling warnings

Similar to Python's warning filter and `-W option` flag, pytest provides its own `-W` flag to control which warnings are ignored, displayed, or turned into errors. See the warning filter documentation for more advanced use-cases.

This code sample shows how to treat any `UserWarning` category class of warning as an error:

```
$ pytest -q test_show_warnings.py -W error::UserWarning
F                                                                        [100%]
================================= FAILURES =================================
_____________________________________ test_one _____________________________________

    def test_one():
>       assert api_v1() == 1
               ^^^^^^^^

test_show_warnings.py:10:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _

    def api_v1():
>       warnings.warn(UserWarning("api v1, should use functions from v2"))
E       UserWarning: api v1, should use functions from v2

test_show_warnings.py:5: UserWarning
```

(continues on next page)

(continued from previous page)

```
======================== short test summary info ========================
FAILED test_show_warnings.py::test_one - UserWarning: api v1, should use ...
1 failed in 0.12s
```

The same option can be set in the configuration file using the *filterwarnings* configuration option. For example, the configuration below will ignore all user warnings and specific deprecation warnings matching a regex, but will transform all other warnings into errors.

```
[pytest]
filterwarnings = [
    'error',
    'ignore::UserWarning',
    # Note the use of single quote below to denote "raw" strings in TOML.
    'ignore:function ham\(\) is deprecated:DeprecationWarning',
]
```

```
[pytest]
filterwarnings =
    error
    ignore::UserWarning
    ignore:function ham\(\) is deprecated:DeprecationWarning
```

When a warning matches more than one option in the list, the action for the last matching option is performed.

**Note**

The `-W` flag and the *filterwarnings* configuration option use warning filters that are similar in structure, but each configuration option interprets its filter differently. For example, *message* in `filterwarnings` is a string containing a regular expression that the start of the warning message must match, case-insensitively, while *message* in `-W` is a literal string that the start of the warning message must contain (case-insensitively), ignoring any whitespace at the start or end of message. Consult the warning filter documentation for more details.

### 2.12.2 `@pytest.mark.filterwarnings`

You can use the *@pytest.mark.filterwarnings* mark to add warning filters to specific test items, allowing you to have finer control of which warnings should be captured at test, class or even module level:

```
import warnings


def api_v1():
    warnings.warn(UserWarning("api v1, should use functions from v2"))
    return 1


@pytest.mark.filterwarnings("ignore:api v1")
def test_one():
    assert api_v1() == 1
```

You can specify multiple filters with separate decorators:

```
# Ignore "api v1" warnings, but fail on all other warnings
@pytest.mark.filterwarnings("ignore:api v1")
@pytest.mark.filterwarnings("error")
def test_one():
    assert api_v1() == 1
```

You can also pass multiple filters to a single mark by providing multiple arguments:

```
# Later arguments take precedence, matching warnings.filterwarnings behavior.
@pytest.mark.filterwarnings("error", "ignore:api v1")
def test_one():
    assert api_v1() == 1
```

**Important**

Regarding decorator order and filter precedence: it's important to remember that decorators are evaluated in reverse order, so you have to list the warning filters in the reverse order compared to traditional `warnings.filterwarnings()` and `-W option` usage. This means in practice that filters from earlier *@pytest.mark.filterwarnings* decorators take precedence over filters from later decorators, as illustrated in the example above.

Filters applied using a mark take precedence over filters passed on the command line or configured by the *filterwarnings* configuration option.

You may apply a filter to all tests of a class by using the *filterwarnings* mark as a class decorator or to all tests in a module by setting the *pytestmark* variable:

```
# turns all warnings into errors for this module
pytestmark = pytest.mark.filterwarnings("error")
```

**Note**

If you want to apply multiple filters (by assigning a list of *filterwarnings* mark to *pytestmark*), you must use the traditional `warnings.filterwarnings()` ordering approach (later filters take precedence), which is the reverse of the decorator approach mentioned above.

*Credits go to Florian Schulze for the reference implementation in the* pytest-warnings *plugin.*

### 2.12.3 Setting a maximum number of warnings

Added in version 9.1.

You can use the *--max-warnings* command-line option to fail the test run if the total number of warnings exceeds a given threshold:

```
pytest --max-warnings=10
```

If all tests pass but the number of warnings exceeds the threshold, pytest will exit with code `6` (*ExitCode* `MAX_WARNINGS_ERROR`). This is useful for gradually ratcheting down warnings in a codebase.

Note that *filtered warnings* do not count toward this maximum total.

The threshold can also be set in the configuration file using *max_warnings*:

```
[pytest]
max_warnings = 10
```

```
[pytest]
max_warnings = 10
```

**Note**

If tests fail, the exit code will be `1` (*`ExitCode`* `TESTS_FAILED`) regardless of the warning count. `MAX_WARNINGS_ERROR` is only reported when all tests pass but the warning threshold is exceeded.

### 2.12.4 Disabling warnings summary

Although not recommended, you can use the *`--disable-warnings`* command-line option to suppress the warning summary entirely from the test run output.

### 2.12.5 Disabling warning capture entirely

This plugin is enabled by default but can be disabled entirely in your configuration file with:

```
[pytest]
addopts = ["-p", "no:warnings"]
```

```
[pytest]
addopts = -p no:warnings
```

Or passing `-p no:warnings` in the command-line. This might be useful if your test suite handles warnings using an external system.

### 2.12.6 DeprecationWarning and PendingDeprecationWarning

By default pytest will display `DeprecationWarning` and `PendingDeprecationWarning` warnings from user code and third-party libraries, as recommended by **PEP 565**. This helps users keep their code modern and avoid breakages when deprecated warnings are effectively removed.

However, in the specific case where users capture any type of warnings in their test, either with *`pytest.warns()`*, *`pytest.deprecated_call()`* or using the *`recwarn`* fixture, no warning will be displayed at all.

Sometimes it is useful to hide some specific deprecation warnings that happen in code that you have no control over (such as third-party libraries), in which case you might use the warning filters options (configuration or marks) to ignore those warnings.

For example:

```
[pytest]
filterwarnings = [
    'ignore:.*U.*mode is deprecated:DeprecationWarning',
]
```

```
[pytest]
filterwarnings =
    ignore:.*U.*mode is deprecated:DeprecationWarning
```

This will ignore all warnings of type `DeprecationWarning` where the start of the message matches the regular expression `".*U.*mode is deprecated"`.

See *@pytest.mark.filterwarnings* and *Controlling warnings* for more examples.

> **Note**
>
> If warnings are configured at the interpreter level, using the `PYTHONWARNINGS` environment variable or the `-W` command-line option, pytest will not configure any filters by default.
>
> Also pytest doesn't follow **PEP 565** suggestion of resetting all warning filters because it might break test suites that configure warning filters themselves by calling `warnings.simplefilter()` (see #2430 for an example of that).

### 2.12.7 Ensuring code triggers a deprecation warning

You can also use *`pytest.deprecated_call()`* for checking that a certain function call triggers a `DeprecationWarning`, `PendingDeprecationWarning` or `FutureWarning`:

```
import pytest


def test_myfunction_deprecated():
    with pytest.deprecated_call():
        myfunction(17)
```

This test will fail if `myfunction` does not issue a deprecation warning when called with a `17` argument.

### 2.12.8 Asserting warnings with the warns function

You can check that code raises a particular warning using *`pytest.warns()`*, which works in a similar manner to *raises* (except that *raises* does not capture all exceptions, only the `expected_exception`):

```
import warnings

import pytest


def test_warning():
    with pytest.warns(UserWarning):
        warnings.warn("my warning", UserWarning)
```

The test will fail if the warning in question is not raised. Use the keyword argument `match` to assert that the warning matches a text or regex. To match a literal string that may contain regular expression metacharacters like `(` or `.`, the pattern can first be escaped with `re.escape`.

Some examples:

```
>>> with warns(UserWarning, match="must be 0 or None"):
...     warnings.warn("value must be 0 or None", UserWarning)
...

>>> with warns(UserWarning, match=r"must be \d+$"):
...     warnings.warn("value must be 42", UserWarning)
...
```

(continues on next page)

(continued from previous page)

```
>>> with warns(UserWarning, match=r"must be \d+$"):
...     warnings.warn("this is not here", UserWarning)
...
Traceback (most recent call last):
  ...
Failed: Regex pattern did not match any of the 1 warnings emitted.
 Regex: ...
 Emitted warnings: ...UserWarning...

>>> with warns(UserWarning, match=re.escape("issue with foo() func")):
...     warnings.warn("issue with foo() func")
...
```

The function also returns a list of all raised warnings (as `warnings.WarningMessage` objects), which you can query for additional information:

```
with pytest.warns(RuntimeWarning) as record:
    warnings.warn("another warning", RuntimeWarning)

# check that only one warning was raised
assert len(record) == 1
# check that the message matches
assert record[0].message.args[0] == "another warning"
```

Alternatively, you can examine raised warnings in detail using the *recwarn* fixture (see *below*).

The *recwarn* fixture automatically ensures to reset the warnings filter at the end of the test, so no global state is leaked.

### 2.12.9 Recording warnings

You can record raised warnings either using the *pytest.warns()* context manager or with the *recwarn* fixture.

To record with *pytest.warns()* without asserting anything about the warnings, pass no arguments as the expected warning type and it will default to a generic Warning:

```
with pytest.warns() as record:
    warnings.warn("user", UserWarning)
    warnings.warn("runtime", RuntimeWarning)

assert len(record) == 2
assert str(record[0].message) == "user"
assert str(record[1].message) == "runtime"
```

The *recwarn* fixture will record warnings for the whole function:

```
import warnings


def test_hello(recwarn):
    warnings.warn("hello", UserWarning)
    assert len(recwarn) == 1
    w = recwarn.pop(UserWarning)
    assert issubclass(w.category, UserWarning)
```

(continues on next page)

(continued from previous page)

```
    assert str(w.message) == "hello"
    assert w.filename
    assert w.lineno
```

Both the *recwarn* fixture and the *pytest.warns()* context manager return the same interface for recorded warnings: a *WarningsRecorder* instance. To view the recorded warnings, you can iterate over this instance, call `len` on it to get the number of recorded warnings, or index into it to get a particular recorded warning.

### 2.12.10 Additional use cases of warnings in tests

Here are some use cases involving warnings that often come up in tests, and suggestions on how to deal with them:

- To ensure that **at least one** of the indicated warnings is issued, use:

```
def test_warning():
    with pytest.warns((RuntimeWarning, UserWarning)):
        ...
```

- To ensure that **only** certain warnings are issued, use:

```
def test_warning(recwarn):
    ...
    assert len(recwarn) == 1
    user_warning = recwarn.pop(UserWarning)
    assert issubclass(user_warning.category, UserWarning)
```

- To ensure that **no** warnings are emitted, use:

```
def test_warning():
    with warnings.catch_warnings():
        warnings.simplefilter("error")
        ...
```

- To suppress warnings, use:

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    ...
```

### 2.12.11 Custom failure messages

Recording warnings provides an opportunity to produce custom test failure messages for when no warnings are issued or other conditions are met.

```
def test():
    with pytest.warns(Warning) as record:
        f()
        if not record:
            pytest.fail("Expected a warning!")
```

If no warnings are issued when calling `f`, then `not record` will evaluate to `True`. You can then call *pytest.fail()* with a custom error message.

### 2.12.12 Internal pytest warnings

pytest may generate its own warnings in some situations, such as improper usage or deprecated features.

For example, pytest will emit a warning if it encounters a class that matches *python_classes* but also defines an `__init__` constructor, as this prevents the class from being instantiated:

```
# content of test_pytest_warnings.py
class Test:
    def __init__(self):
        pass

    def test_foo(self):
        assert 1 == 1
```

```
$ pytest test_pytest_warnings.py -q

============================= warnings summary ==============================
test_pytest_warnings.py:1
  /home/sweet/project/test_pytest_warnings.py:1: PytestCollectionWarning: cannot␣
↪collect test class 'Test' because it has a __init__ constructor (from: test_pytest_
↪warnings.py)
    class Test:

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1 warning in 0.12s
```

These warnings might be filtered using the same builtin mechanisms used to filter other types of warnings.

Please read our *Backwards Compatibility Policy* to learn how we proceed about deprecating and eventually removing features.

The full list of warnings is listed in *the reference documentation*.

### 2.12.13 Resource Warnings

Additional information of the source of a `ResourceWarning` can be obtained when captured by pytest if `tracemalloc` module is enabled.

One convenient way to enable `tracemalloc` when running tests is to set the `PYTHONTRACEMALLOC` to a large enough number of frames (say `20`, but that number is application dependent).

For more information, consult the Python Development Mode section in the Python documentation.

## 2.13 How to use skip and xfail to deal with tests that cannot succeed

You can mark test functions that cannot be run on certain platforms or that you expect to fail so pytest can deal with them accordingly and present a summary of the test session, while keeping the test suite *green*.

A **skip** means that you expect your test to pass only if some conditions are met, otherwise pytest should skip running the test altogether. Common examples are skipping windows-only tests on non-windows platforms, or skipping tests that depend on an external resource which is not available at the moment (for example a database).

An **xfail** means that you expect a test to fail for some reason. A common example is a test for a feature not yet implemented, or a bug not yet fixed. When a test passes despite being expected to fail (marked with `pytest.mark.xfail`), it's an **xpass** and will be reported in the test summary.

`pytest` counts and lists *skip* and *xfail* tests separately. Detailed information about skipped/xfailed tests is not shown by default to avoid cluttering the output. You can use the `-r` option to see details corresponding to the "short" letters shown in the test progress:

```
pytest -rxXs  # show extra info on xfailed, xpassed, and skipped tests
```

More details on the `-r` option can be found by running `pytest -h`.

(See *Builtin configuration file options*)

### 2.13.1 Skipping test functions

The simplest way to skip a test function is to mark it with the `skip` decorator which may be passed an optional `reason`:

```
@pytest.mark.skip(reason="no way of currently testing this")
def test_the_unknown(): ...
```

Alternatively, it is also possible to skip imperatively during test execution or setup by calling the `pytest.skip(reason)` function:

```
def test_function():
    if not valid_config():
        pytest.skip("unsupported configuration")
```

The imperative method is useful when it is not possible to evaluate the skip condition during import time.

It is also possible to skip the whole module using `pytest.skip(reason, allow_module_level=True)` at the module level:

```
import sys

import pytest

if not sys.platform.startswith("win"):
    pytest.skip("skipping windows-only tests", allow_module_level=True)
```

**Reference**: *pytest.mark.skip*

#### skipif

If you wish to skip something conditionally then you can use `skipif` instead. Here is an example of marking a test function to be skipped when run on an interpreter earlier than Python3.13:

```
import sys


@pytest.mark.skipif(sys.version_info < (3, 13), reason="requires python3.13 or higher
↪")
def test_function(): ...
```

If the condition evaluates to `True` during collection, the test function will be skipped, with the specified reason appearing in the summary when using `-rs`.

You can share `skipif` markers between modules. Consider this test module:

```
# content of test_mymodule.py
import mymodule

minversion = pytest.mark.skipif(
    mymodule.__versioninfo__ < (1, 1), reason="at least mymodule-1.1 required"
)


@minversion
def test_function(): ...
```

You can import the marker and reuse it in another test module:

```
# test_myothermodule.py
from test_mymodule import minversion


@minversion
def test_anotherfunction(): ...
```

For larger test suites it's usually a good idea to have one file where you define the markers which you then consistently apply throughout your test suite.

Alternatively, you can use *condition strings* instead of booleans, but they can't be shared between modules easily so they are supported mainly for backward compatibility reasons.

**Reference**: *pytest.mark.skipif*

### Skip all test functions of a class or module

You can use the `skipif` marker (as any other marker) on classes:

```
@pytest.mark.skipif(sys.platform == "win32", reason="does not run on windows")
class TestPosixCalls:
    def test_function(self):
        "will not be setup or run under 'win32' platform"
```

If the condition is `True`, this marker will produce a skip result for each of the test methods of that class.

If you want to skip all test functions of a module, you may use the *pytestmark* global:

```
# test_module.py
pytestmark = pytest.mark.skipif(...)
```

If multiple `skipif` decorators are applied to a test function, it will be skipped if any of the skip conditions is true.

### Skipping files or directories

Sometimes you may need to skip an entire file or directory, for example if the tests rely on Python version-specific features or contain code that you do not wish pytest to run. In this case, you must exclude the files and directories from collection. Refer to *Customizing test collection* for more information.

### Skipping on a missing import dependency

You can skip tests on a missing import by using *pytest.importorskip* at module level, within a test, or test setup function.

```
docutils = pytest.importorskip("docutils")
```

If `docutils` cannot be imported here, this will lead to a skip outcome of the test. You can also skip based on the version number of a library:

```
docutils = pytest.importorskip("docutils", minversion="0.3")
```

The version will be read from the specified module's `__version__` attribute.

### Summary

Here's a quick guide on how to skip tests in a module in different situations:

1. Skip all tests in a module unconditionally:

   ```
   pytestmark = pytest.mark.skip("all tests still WIP")
   ```

2. Skip all tests in a module based on some condition:

   ```
   pytestmark = pytest.mark.skipif(sys.platform == "win32", reason="tests for␣
   ↪linux only")
   ```

3. Skip all tests in a module if some import is missing:

   ```
   pexpect = pytest.importorskip("pexpect")
   ```

## 2.13.2 XFail: mark test functions as expected to fail

You can use the `xfail` marker to indicate that you expect a test to fail:

```
@pytest.mark.xfail
def test_function(): ...
```

This test will run but no traceback will be reported when it fails. Instead, terminal reporting will list it in the "expected to fail" (`XFAIL`) or "unexpectedly passing" (`XPASS`) sections.

Alternatively, you can also mark a test as `XFAIL` from within the test or its setup function imperatively:

```
def test_function():
    if not valid_config():
        pytest.xfail("failing configuration (but should work)")
```

```
def test_function2():
    import slow_module

    if slow_module.slow_function():
        pytest.xfail("slow_module taking too long")
```

These two examples illustrate situations where you don't want to check for a condition at the module level, which is when a condition would otherwise be evaluated for marks.

This will make `test_function` `XFAIL`. Note that no other code is executed after the *`pytest.xfail()`* call, differently from the marker. That's because it is implemented internally by raising a known exception.

**Reference**: *pytest.mark.xfail*

#### `condition` parameter

If a test is only expected to fail under a certain condition, you can pass that condition as the first parameter:

```
@pytest.mark.xfail(sys.platform == "win32", reason="bug in a 3rd party library")
def test_function(): ...
```

Note that you have to pass a reason as well (see the parameter description at *pytest.mark.xfail*).

#### `reason` parameter

You can specify the motive of an expected failure with the `reason` parameter:

```
@pytest.mark.xfail(reason="known parser issue")
def test_function(): ...
```

#### `raises` parameter

If you want to be more specific as to why the test is failing, you can specify a single exception, or a tuple of exceptions, in the `raises` argument.

```
@pytest.mark.xfail(raises=RuntimeError)
def test_function(): ...
```

Then the test will be reported as a regular failure if it fails with an exception not mentioned in `raises`.

#### `run` parameter

If a test should be marked as xfail and reported as such but should not be even executed, use the `run` parameter as `False`:

```
@pytest.mark.xfail(run=False)
def test_function(): ...
```

This is particularly useful for xfailing tests that are crashing the interpreter and should be investigated later.

#### `strict` parameter

Both `XFAIL` and `XPASS` don't fail the test suite by default. You can change this by setting the `strict` keyword-only parameter to `True`:

```
@pytest.mark.xfail(strict=True)
def test_function(): ...
```

This will make `XPASS` ("unexpectedly passing") results from this test to fail the test suite.

You can change the default value of the `strict` parameter using the `strict_xfail` ini option:

```
[pytest]
xfail_strict = true
```

```
[pytest]
strict_xfail = true
```